

**СПЕЦИФИЧНЫЕ ДЛЯ АЛГОРИТМИЧЕСКОГО ПАТТЕРНА
СТРУКТУРНЫЕ ПОДСКАЗКИ В АВТОМАТИЧЕСКОЙ
ЛОКАЛИЗАЦИИ ЛОГИЧЕСКИХ ОШИБОК
С ПОМОЩЬЮ БОЛЬШОЙ ЯЗЫКОВОЙ МОДЕЛИ**

Предварительно зарегистрированный калибровочный эксперимент
на корпусе ConDefects-Python
Исследование выполнено в рамках проекта AlgoRythm

Авторы:
Сәдібек Нұрмұхамбет, 11 «А»
Хасан Амир, 11 «Е»
Научный руководитель: Исказиева А.Ж.

Направление: [НАПРАВЛЕНИЕ]
Секция: [СЕКЦИЯ]

Атырау, 2026

АННОТАЦИЯ

Цель. Измерить, повышает ли специфичная для алгоритмического паттерна структурная подсказка точность локализации логической ошибки языковой моделью по сравнению с сопоставимой по длине и формату общей отладочной подсказкой.

Гипотеза. Доля программ, в которых модель ставит истинно ошибочную строку первой, при подсказке по паттерну выше, чем при общей подсказке.

Этапы и методика. Обзор литературы; построение воспроизводимой выборки ($n = 30$) из корпуса ConDefects-Python механическими фильтрами; независимая слепая разметка паттернов двумя моделями (каппа Коэна ≈ 0.752269); заморозка протокола, подсказок, параметров модели и плана анализа до сбора данных; 600 обращений к одной замороженной модели при температуре 0 (пять повторений, четыре условия, две парные сессии); точный критерий Макнемара, бутстреп-интервалы, проверка нулевым контролем.

Новизна. Эффект информации о паттерне изолирован от эффекта получения любого дополнительного отладочного текста: подсказка по паттерну сравнивалась не с её отсутствием, а с согласованной по длине, структуре и формату общей подсказкой.

Самостоятельность. Автором определены тема, исследовательский вопрос, гипотеза, дизайн с согласованным контролем и границы выводов; использование инструментов ИИ раскрыто в Приложении Д.

Результаты. Первичная гипотеза не подтверждена: Топ-1 = 12/30 (40.00%) в обоих условиях, разница 0.00 п.п., $p = 1.0$, 95% ДИ $[-16.67; +16.67]$ п.п. Вторичные описательные показатели направлены в пользу подсказки по паттерну (Топ-3 60.00% против 50.00%; Топ-5 63.33% против 53.33%; EXAM* 0.445055 против 0.510798), но значимости не достигают.

Выводы. Результат не свидетельствует ни об эквивалентности условий, ни об отсутствии эффекта; требуется подтверждающее исследование большего объёма. Для платформы AlgoRythm оснований внедрять обусловленного паттерном отладочного помощника пока нет.

Ключевые слова: локализация ошибок, большая языковая модель, алгоритмические паттерны, структурные подсказки, отладка.

АҢДАТПА

Мақсаты. Алгоритмдік паттернге тән құрылымдық нұсқау тілдік модельдің логикалық қате орнын анықтау дәлдігін ұзындығы мен пішімі бойынша сәйкестендірілген жалпы жөндеу нұсқауымен салыстырғанда арттыра ма — соны өлшеу.

Болжамы. Модель шын мәнінде қате жолды бірінші орынға қоятын бағдарламалар үлесі паттерн бойынша нұсқауда жалпы нұсқауға қарағанда жоғары.

Кезеңдері мен әдістемесі. Әдебиетке шолу; ConDefects-Python корпусынан механикалық сүзгілермен қайта жаңғыртылатын іріктеме ($n = 30$) құру; паттерндерді екі модельдің тәуелсіз әрі жасырын таңбалауы (Коэн каппасы ≈ 0.752269); деректер жиналғанға дейін хаттаманы, нұсқауларды, модель параметрлерін және талдау жоспарын бекіту; нөлдік температурада бір бекітілген модельге 600 сұрау (бес қайталау, төрт шарт, екі жұптық сессия); Макнемардың дәл критерийі, бутстреп аралықтары, нөлдік бақылау.

Жаңалығы. Паттерн туралы ақпараттың әсері кез келген қосымша жөндеу мәтінін алу әсерінен ажыратылған: паттерн бойынша нұсқау оның болмауымен емес, ұзындығы, құрылымы және пішімі бойынша сәйкестендірілген жалпы нұсқаумен салыстырылды.

Дербестігі. Тақырыпты, зерттеу сұрағын, болжамды, сәйкестендірілген бақылауы бар дизайнды және қорытынды шекараларын автор айқындады; жасанды интеллект құралдарының қолданылуы Д қосымшасында ашылған.

Нәтижелері. Бастапқы болжам расталмады: $\text{Top-1} = 12/30$ (40.00%) екі шартта да, айырмасы 0.00 п.т., $p = 1.0$, 95% сенімділік аралығы $[-16.67; +16.67]$ п.т. Қосымша сипаттамалық көрсеткіштер паттерн бойынша нұсқау пайдасына бағытталған (Top-3 60.00% пен 50.00%; Top-5 63.33% пен 53.33%; EXAM* 0.445055 пен 0.510798), бірақ маңыздылыққа жетпейді.

Қорытындысы. Нәтиже шарттардың баламалылығын да, әсердің жоқтығын да дәлелдемейді; көлемі үлкенірек растаушы зерттеу қажет. AlgoRythm платформасы үшін паттернге негізделген жөндеу көмекшісін енгізуге негіз әзірге жоқ.

Түйін сөздер: қатені локализациялау, үлкен тілдік модель, алгоритмдік паттерндер, құрылымдық нұсқаулар, жөндеу.

ABSTRACT

Objective. To measure whether an algorithm-pattern-specific structural prior improves a language model’s fault-localization accuracy relative to a length- and format-matched generic prior.

Hypothesis. The share of programs in which the model ranks the truly faulty line first is higher under the pattern prior than under the generic one.

Stages and methodology. Literature review; a reproducible 30-program sample built from ConDefects-Python by mechanical filters; independent blinded pattern annotation by two models ($\kappa \approx 0.752269$); pre-registration of protocol, priors, parameters and analysis plan; 600 calls to one frozen model at temperature 0 (five repetitions, four conditions, two paired sessions); exact McNemar test, bootstrap intervals, null-control check.

Novelty. The effect of pattern information is isolated from that of receiving any extra debugging text: the pattern prior was compared against a length-, structure- and format-matched generic prior rather than against no prior.

Contribution. The author set the topic, research question, hypothesis, matched-control design and inference boundaries; AI-tool use is disclosed in Appendix D.

Results. The primary hypothesis was not supported: Top-1 = 12/30 (40.00%) in both conditions (difference 0.00 pp), $p = 1.0$, 95% CI $[-16.67, +16.67]$ pp. Secondary descriptive measures favour the pattern prior (Top-3 60.00% vs 50.00%; Top-5 63.33% vs 53.33%; EXAM* 0.445055 vs 0.510798) but reach no significance.

Conclusions. The result is evidence neither of equivalence nor of no effect; a larger confirmatory study is needed. For AlgoRhythm there is no basis yet for deploying a pattern-conditioned debugging assistant.

Keywords: fault localization, large language model, algorithmic patterns, structural priors, debugging.

ОГЛАВЛЕНИЕ

АННОТАЦИЯ	2
АНДАТПА	3
ABSTRACT	4
1. ВВЕДЕНИЕ	6
1.1 Актуальность исследования	6
1.2 Проблема исследования	6
1.3 Исследовательский вопрос	6
1.4 Цель исследования	7
1.5 Задачи исследования	7
1.6 Гипотеза	7
1.7 Научная новизна	7
1.8 Практическая значимость	7
2. ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ	8
2.1 Литературный обзор	8
2.2 Исследовательский пробел	9
2.3 Общий дизайн исследования	10
2.4 Набор данных ConDefects-Python	11
2.5 Формирование выборки	11
2.6 Определение алгоритмических паттернов	13
2.7 Надёжность разметки	13
2.8 Итоговая экспериментальная выборка	14
2.9 Экспериментальные условия	14
2.10 Модель и параметры эксперимента	16
2.11 Метрики оценки	17
2.12 Статистический анализ	17
2.13 Результаты	18
2.14 Анализ результатов	22
2.15 Ограничения исследования	24
2.16 Практическое значение для AlgoRythm	25
2.17 Направления будущих исследований	26
3. ЗАКЛЮЧЕНИЕ	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28
ПРИЛОЖЕНИЯ	30
Приложение А. Тексты экспериментальных подсказок	30
Приложение Б. Полная воронка формирования выборки	31
Приложение В. Дополнительные статистические результаты	31
Приложение Г. Согласие аннотаторов по классам паттернов	32
Приложение Д. Воспроизводимость, конфигурация и заявление об использовании инструментов ИИ	32

1. ВВЕДЕНИЕ

1.1 Актуальность исследования

Когда программа начинающего не работает, у ошибки есть два принципиально разных вида. Синтаксическую ошибку находит компилятор: он называет файл, строку и характер нарушения. Логическая ошибка ведёт себя иначе — программа успешно запускается и выдаёт ответ, но ответ неверен; сообщения об ошибке не появляется, и искать место сбоя приходится самому.

Именно этот второй вид ошибок и составляет основную трудность. Классическая модель трудностей обучения программированию [1] выделяет отладку в отдельную, систематически недообучаемую компетенцию, а многоинституциональное исследование [2] описывает отладку новичков словом *flailing* — «барахтанье»: неспланированные пробы без гипотез. Причина установлена ещё в 1980-х: большинство ошибок новичков — не непонимание синтаксиса, а сбой на уровне композиции замысла [3]. Существующие каталоги логических ошибок [4] при этом организованы независимо от алгоритма: «выход за границу цикла» числится одним типом ошибки и для двоичного поиска, и для заполнения таблицы динамического программирования — хотя проявляется он в этих случаях по-разному.

1.2 Проблема исследования

Автоматическая **локализация ошибки** (fault localization) — задача указать разработчику строки программы, которые вероятнее всего содержат дефект, упорядочив их по убыванию подозрительности. Классические методы разрабатывались для промышленного кода и на коде новичков работают заметно хуже [5]. Большие языковые модели дают лучший результат: сравнение тринадцати моделей на трёх наборах данных с ошибками начинающих показало, что рассуждающие модели превосходят классические подходы почти по всем метрикам [6].

Общим для всех этих систем является то, что **ни одна из них не получает информации об алгоритмическом замысле программы**. Между тем два вложенных цикла могут быть полным перебором всех пар элементов, а могут быть заполнением таблицы динамического программирования: синтаксически эти фрагменты почти неразличимы, но правдоподобные места ошибки в них разные — границы диапазона и порядок операций формулы в первом случае, значение базового случая и направление заполнения таблицы во втором. Локализатор, не знающий, какой из двух замыслов перед ним, вынужден рассматривать оба множества гипотез сразу. При этом технология, способная сообщить недостающую информацию, существует и зрела: распознавание алгоритма по исходному коду достигает F1-меры около 95% на структурных признаках [7]. Две области просто не были соединены и проверены вместе.

1.3 Исследовательский вопрос

Повышает ли информация о структурном алгоритмическом паттерне точность локализации логической ошибки языковой моделью по сравнению с сопоставимой по длине и формату общей отладочной подсказкой?

Проверяемая причинная цепочка: **алгоритмический замысел → структурная подсказка → локализация логической ошибки**.

Принципиально, что это **не** вопрос «помогает ли подсказка по сравнению с её отсутствием». При таком переходе одновременно меняются как минимум пять вещей: содержание о паттерне, объём текста, появление нумерованного списка, директива «проверь это сначала» и сам сигнал о том, что у программы есть узнаваемая структура; любое улучшение объяснялось бы тогда альтернативной гипотезой «любая структурированная инструкция помогает». Поэтому сравнение ведётся с **согласованной общей подсказкой**, удерживающей все пять факторов постоянными и варьирующей ровно одно: выведено ли содержание подсказки из паттерна данной программы.

1.4 Цель исследования

Количественно измерить вклад структурной информации об алгоритмическом паттерне в точность автоматической локализации логической ошибки, изолировав этот вклад от эффекта самого факта наличия дополнительной инструкции.

1.5 Задачи исследования

1. Построить воспроизводимую выборку программ с ровно одной подтверждённой логической ошибкой из корпуса ConDefects-Python.
2. Зафиксировать словарь алгоритмических паттернов и получить метки процедурой с измеримой надёжностью.
3. Разработать пару подсказок — по паттерну и общую, — механически согласованных по объёму, формату и тону.
4. Зафиксировать протокол, параметры модели и план анализа до сбора данных.
5. Выполнить эксперимент и вычислить предварительно зарегистрированные показатели.
6. Оценить границы интерпретации результата и сформулировать следующие шаги.

1.6 Гипотеза

- **H1 (первичная, направленная).** Доля программ, в которых истинно ошибочная строка занимает первое место в выдаче модели (Top-1), будет выше при подсказке по паттерну, чем при согласованной общей подсказке.
- **H0 (нулевая гипотеза).** Среди программ, по которым два условия расходятся, успехи распределены между ними не иначе, чем случайно.
- **Вторичное ожидание.** Оба условия с подсказкой превзойдут условие без блока подсказки.
- **Калибровочное ожидание.** Одно и то же контрольное условие, выполненное дважды независимо при температуре 0, разойдётся с самим собой почти на ноль.

Гипотезы приведены в том виде, в каком были зафиксированы до сбора данных.

1.7 Научная новизна

Новизна работы — **не** в применении языковой модели к поиску ошибок и **не** в самой идее опираться при отладке на алгоритмический тип программы. Она состоит в **изоляции эффекта паттерн-специфичности**: эксперимент отделяет действие структурной информации, привязанной к алгоритмическому классу, от действия самого факта получения дополнительного текста. Условие с подсказкой по паттерну сравнивается с согласованной по длине, структуре и формату общей подсказкой, и именно это сравнение объявлено первичным. Вопрос тем самым переводится из инженерного («помогает ли более длинный запрос») в научный («несёт ли информация об алгоритмическом замысле измеримую ценность»). Работа соединяет при этом две зрелые, но не связанные области: ни одна система до сих пор не подавала распознанный алгоритмический класс на вход локализатору.

1.8 Практическая значимость

Исследование выполнено в рамках проекта **AlgoRythm** — учебной платформы, сочетающей разбор алгоритмических паттернов с песочницей для кода. Если информация о паттерне измеримо помогает находить ошибку, в платформе имеет смысл строить обусловленного паттерном отладочного помощника; если нет — внедрять его под видом обоснованного решения нельзя. При этом **никакого утверждения об эффективности платформы или о результатах обучения здесь не делается**: ни один человек-испытуемый в исследовании не участвовал.

2. ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ

2.1 Литературный обзор

Постановка задачи локализации. Дана программа, которая проходит часть тестов и не проходит остальные; требуется выдать список строк, упорядоченный по убыванию вероятности того, что именно эта строка содержит дефект. Качество измеряют долей случаев, в которых настоящая ошибочная строка попала в первые K позиций. Исторически задача выросла из диагностики систематических ошибок вообще: классическая работа о «процедурных ошибках» в школьной арифметике [8] показала, что ошибки обучающихся не случайны, а порождаются устойчивыми неверными процедурами, и что их можно каталогизировать и распознавать.

Спектральная локализация (spectrum-based fault localization, SBFL) вычисляет «подозрительность» каждой строки из того, насколько часто она выполняется в проваленных тестах и насколько редко — в пройденных. Строка, исполняемая всеми провальными тестами и ни одним успешным, получает максимальную оценку. Формулы Tarantula [9], Ochiai и DStar [10] различаются способом свёртки этих четырёх чисел; обзор более чем 150 работ этого семейства дан в [11]. Для кода начинающих метод плохо применим по двум эмпирически установленным причинам [12]: значительная доля программ новичков нарушает базовое условие применимости — нужен хотя бы один пройденный и хотя бы один проваленный тест, — а короткие программы порождают множество строк с одинаковой подозрительностью, которые метод различить не может.

Мутационная локализация (mutation-based fault localization, MBFL) вносит в каждую строку небольшое искусственное изменение — мутацию — и смотрит, начинают ли проходить ранее провальные тесты. Строка, «починяемая» мутациями, признаётся подозрительной. В основе метода лежит гипотеза компетентного программиста [13]: предполагается, что программа почти правильна и отличается от правильной на одну небольшую правку. Для студенческого кода это допущение выполняется хуже: работа по ремонту программ через сопоставление с корректными решениями других обучающихся [14] прямо отмечает, что их решения часто «серьёзно неверны», а не отличаются на одну правку. Наиболее прямой количественный ориентир даёт применение мутационной локализации к 150 программам на Python и 150 на Java из корпуса ConDefects — того же корпуса, который используется здесь: Топ-5 около 61% на Python при затратах порядка 87 машино-часов на 150 программ [5].

Отдельная линия работ использует в качестве подсказки *эталонное решение*: система VsusFL [15] отслеживает значения переменных при выполнении и сопоставляет их с единственным правильным решением той же задачи, а обучаемый подход [16] предсказывает ошибочные строки нейросетевой моделью, работающей со структурой программы. Подсказкой здесь служит конкретная другая программа или модель, обученная на конкретном наборе, а не переносимый, не привязанный к задаче класс алгоритма — именно это различие проверяется в настоящем исследовании.

Большие языковые модели в отладке. Большая языковая модель (large language model, LLM) — нейросеть, обученная на больших объёмах текста и кода и способная по текстовому описанию задачи порождать текстовый ответ. Для локализации ошибки модели подают исходный код и просят вернуть упорядоченный список подозрительных строк. Наиболее прямой предшественник настоящей работы [6] охватил тринадцать моделей и три набора данных с ошибками начинающих и установил, что рассуждающие модели превосходят и спектральную, и мутационную локализацию почти по всем наборам данных и метрикам, но точность каждой модели падает с ростом сложности задачи. Параллельно показано, что GPT-4 значимо превосходит выборку из 964 студентов вводного курса в обнаружении логических ошибок [17], а выявление логических ошибок в студенческих программах автоматизировано на масштабе, достаточном для анализа заблуждений [18].

Существенно, что **ни одна из перечисленных систем не получает информации о том, какую алгоритмическую технику реализует проверяемая программа.** Более того, работа [6] содержит абляцию компонентов запроса — по отдельности добавлялись указание роли, самообъяснение замысла, обоснование, цепочка рассуждений и инструкция сортировки, — и ни один из проверенных компонентов не был обусловлен алгоритмическим классом программы. Методологически настоящее исследование повторяет схему этой абляции, но манипулирует ровно тем компонентом, которого там не было.

Алгоритмические паттерны и структурная информация. Распознавание алгоритма по исходному коду — в значительной степени решённая задача классификации. Принятые решения из открытого онлайн-судьи классифицируются на шесть-семь категорий алгоритмов только по структурным признакам — счётчикам циклов, ветвлений, операторов, без имён переменных, — с F1-мерой около 95–97% [7]. Система AlDeSCo распознаёт алгоритмические мотивы через вручную написанные шаблоны по дереву разбора программы и получает F1 = 0.74 против 0.35 у языковой модели общего назначения на той же задаче [19].

Теоретическое основание для использования этой информации даёт психология понимания программ. Экспертное чтение кода описано как порождение и проверка гипотез о целях программы, запускаемых *маяками* — характерными фрагментами, сигнализирующими о знакомой идиоме [20]. Опытный читатель строит не только представление о потоке управления, но и *модель ситуации* — понимание того, что программа пытается сделать, — тогда как новичок систематически застревает на первом уровне [21, 22]. Алгоритмический паттерн — это факт уровня модели ситуации, и он задаёт конкретный набор гипотез о вероятных местах ошибки. Косвенное подтверждение даёт эксперимент с опытными программистами [23]: даже у них трудности с рекурсией концентрируются вокруг распознавания базового случая, то есть места ошибки действительно группируются по алгоритмической структуре, а не равномерно по коду.

Идея компенсировать неполноту внутренней модели явным описанием более высокого уровня имеет длинную родословную: литература о «мысленных машинах» [24, 25, 26] утверждает, что явная формулировка предполагаемой модели выполнения помогает обучающемуся. Ближайший прямой предок настоящей работы — система PROUST [27], диагностировавшая ошибки новичков на Pascal сопоставлением кода студента с вручную составленной библиотекой ожидаемых *планов* для конкретного задания. Библиотека PROUST составлялась под каждое задание вручную, система не оценивалась количественной метрикой локализации и предшествует и современным классификаторам, и языковым моделям примерно на четыре десятилетия. Родственная, но иначе устроенная система Whyline [28] позволяет спрашивать «почему программа сделала это» и отвечает трассами выполнения; структурная подсказка в настоящей работе, напротив, ограничивает то, *какие* вопросы имеет смысл задать первыми, исходя из алгоритмического типа программы.

Показательно, наконец, что интуиция преподавателей здесь ненадёжна: анализ 37 миллионов компиляций [29] обнаружил, что представления педагогов о самых частых ошибках новичков расходятся с реальной статистикой. Систематика трудностей обучения по-прежнему строится вокруг конструкций языка, а не вокруг реализуемого алгоритма [30], что подтверждают и специализированные инвентари логических ошибок [31], и описания языконезависимых концептуальных ошибок [32]. Наконец, разрыв между способностью проследить выполнение кода по шагам и способностью объяснить, для чего этот код нужен, у новичков устойчив [33], а обзор трёх десятилетий работ по обучению отладке [34] показывает, что явно отладке почти не учат.

2.2 Исследовательский пробел

Обзор приводит к одному узлу, в котором линии не соединены.

Первая линия — распознавание. Алгоритмический паттерн надёжно определяется по исходному коду с F1 около 95% [7, 19]. Все работы этой линии останавливаются на распознавании: ни одна не подаёт распознанную метку на вход следующему шагу рассуждения, и их собственные разделы о будущей работе такого шага не предлагают.

Вторая линия — локализация. Сильнейшие современные локализаторы для кода новичков — спектральные, мутационные [5] и основанные на языковых моделях [6] — не учитывают паттерн: ни один не получает на входе классификацию алгоритмической техники.

Третья линия — интеллектуальный прецедент. Идея диагностировать ошибку через знание о предполагаемом плане реализована в PROUST [27], но под конкретное задание, вручную и без количественной оценки против базовых методов.

Пробел формулируется так: **ни в одной опубликованной работе классифицированный алгоритмический паттерн программы не подаётся на вход системе локализации ошибок, и ни в одной не измерено, даёт ли такая структурная информация выигрыш сверх того, который даёт любая сопоставимая по объёму и формату отладочная инструкция, не связанная с паттерном.** Именно этот непроверенный переход — и именно с согласованным контролем — закрывает настоящее исследование.

2.3 Общий дизайн исследования

Исследование построено как **внутригрупповой эксперимент**: все условия проверяются на одних и тех же тридцати программах, поэтому различия между условиями не могут объясняться различиями в сложности программ.

Каждая из тридцати программ подавалась модели в трёх вариантах запроса, различающихся **только одним вставленным блоком текста**: без блока подсказки, с общей подсказкой и с подсказкой по паттерну. Всё остальное — системная инструкция, шаблон сообщения, нумерация строк исходного кода, требуемый формат ответа — оставалось побайтово идентичным. Общая схема проверяемой цепочки показана на Рисунке 1.

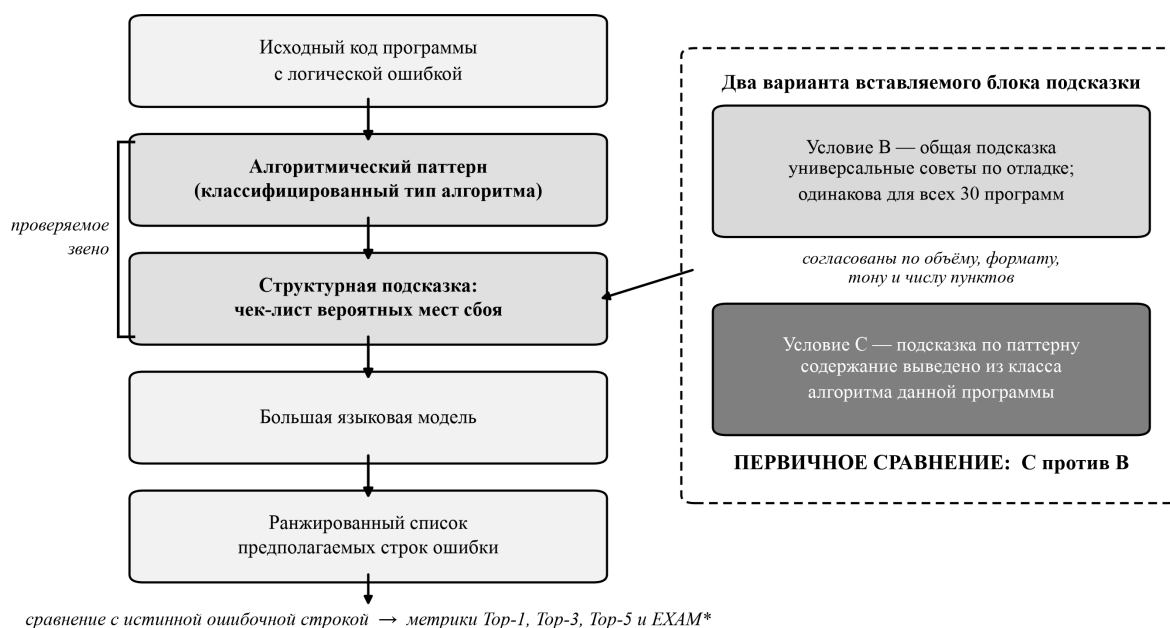


Рисунок 1 — Концептуальная схема исследования. Выделенное звено «алгоритмический паттерн → структурная подсказка» является предметом проверки; остальные звенья цепочки в эксперименте зафиксированы.

Инструментарий проекта выполняет за один запуск ровно два условия, поэтому три условия реализованы как **две парные сессии** на одной и той же выборке: сессия G сравнивала контроль без подсказки с общей подсказкой, сессия P — контроль без подсказки с подсказкой по паттерну. Порядок сессий (сначала G) был определён однократным случайным подбрасыванием до первого обращения к модели.

Все протокольные документы, выборка, тексты подсказок и план статистического анализа были **зафиксированы в неизменяемом виде до появления каких-либо экспериментальных данных**. Это означает, что ни одно правило анализа не могло быть

выбрано после того, как стал известен результат. Подробности фиксации и воспроизводимости приведены в Приложении Д.

2.4 Набор данных ConDefects-Python

Источником данных служит **ConDefects** — исследовательский корпус для локализации ошибок, собранный из реальных решений на Python, отправленных на соревнования по программированию AtCoder. Каждое отклонённое решение сопоставлено с последующим принятым решением того же автора и с официальными тестовыми данными. Разница между «ошибочной» и «исправленной» версиями даёт объективное, не зависящее от мнения исследователя указание на место ошибки.

Корпус выбран по трём причинам: он открыт и лицензирован для исследовательского использования; ошибки в нём настоящие, а не внесённые искусственно; и по нему опубликована количественная оценка классических методов [5], с которой можно соотносить порядок величин.

Одно решение принято сознательно и имеет цену. **Условия задач AtCoder были полностью исключены из всех запросов к модели**, во всех условиях одинаково. Формулировка задачи часто прямо называет предполагаемый алгоритм («найдите за $O(\log n)$...»), и её включение означало бы утечку информации о паттерне в контрольные условия, уничтожив измеряемый контраст. Цена решения — менее реалистичная постановка: реальный отладочный помощник имел бы доступ к условию задачи.

2.5 Формирование выборки

Выборка строилась строго упорядоченным, механически проверяемым конвейером фильтров, применённым до какого-либо содержательного суждения. Из полного корпуса в 2864 ошибочных решения по 985 задачам последовательно исключались: программы с синтаксической ошибкой (1); программы длиной вне диапазона 25–300 строк (1539); программы, чьё исправление затрагивает не одну строку (316); программы, не удовлетворяющие динамическому условию «есть хотя бы один пройденный и хотя бы один проваленный тест на ошибочной версии, и все тесты пройдены на исправленной» (705); и, наконец, повторы по одной задаче — из нескольких решений одной задачи посеянным случайным отбором оставлялось ровно одно (71). Осталась **рамка из 232 программ**, сформированная до какой-либо разметки паттернов. Полная воронка показана на Рисунке 2 и продублирована счётчиками в Приложении Б.

Правило «одна программа на задачу» существенно: решения одной задачи разделяют формулировку, предполагаемую технику и часто сам режим ошибки, а применяемые статистические тесты считают каждую программу независимым наблюдением. Обеспечение уникальности на уровне задачи делает это допущение защитимым.

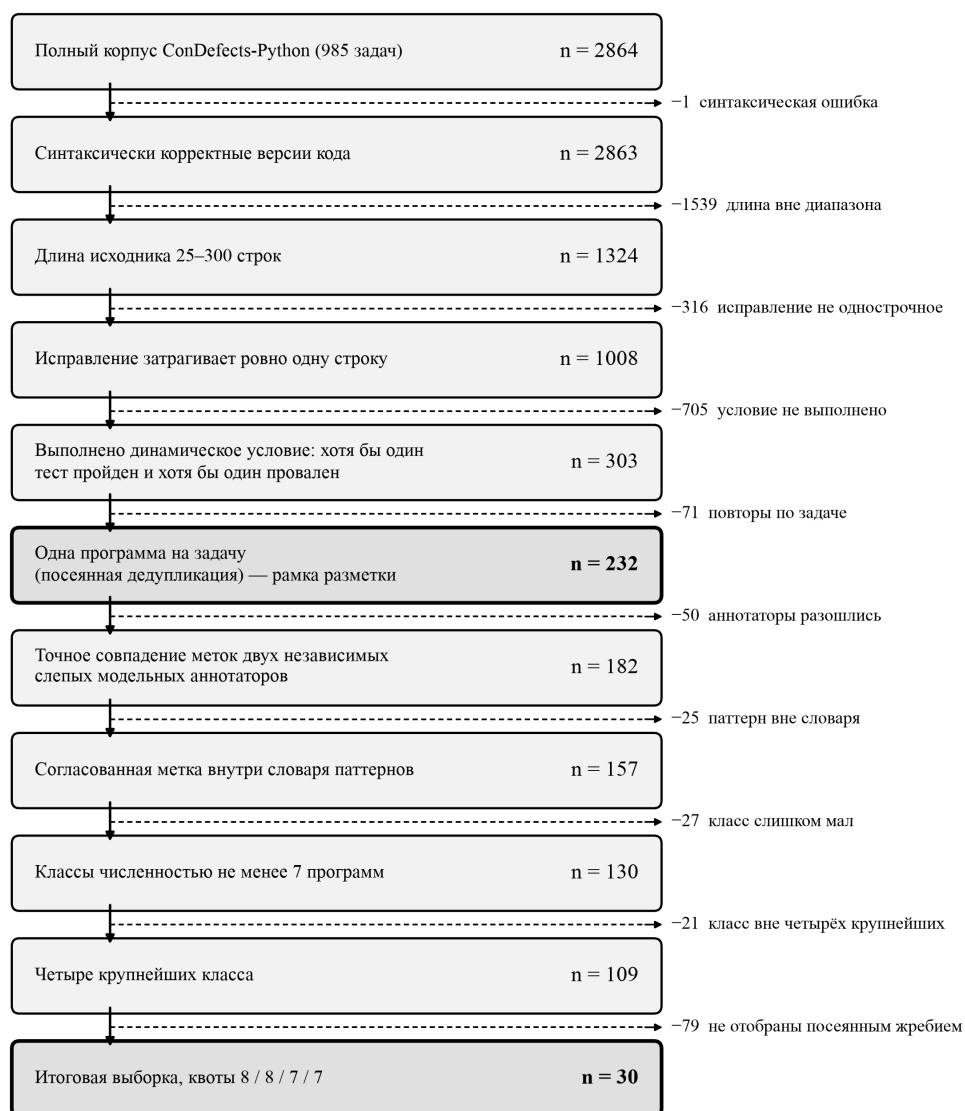


Рисунок 2 — Формирование экспериментальной выборки. Счётчики соответствуют замороженным аудитам отбора; выделены два ключевых узла — рамка независимой разметки ($n = 232$) и итоговая выборка ($n = 30$).

Истинное место ошибки (ground truth) — строка, относительно которой оценивается ответ модели, — определялось по замороженному протоколу, специально построенному так, чтобы содержание подсказок не могло повлиять на ту истину, относительно которой они затем оцениваются. Протокол требует трёх вещей. Во-первых, ошибочная строка выводится **только** из различия между ошибочной и исправленной версиями и данных о проваленных тестах. Во-вторых, решение о месте ошибки записывается в файл, доступный только для однократной записи, и **замораживается до того**, как кто-либо приступит к рассуждению об алгоритмическом паттерне программы. В-третьих, определение паттерна выполняется позже и в условиях, где место ошибки, исправленный код и различие между версиями недоступны.

Формы исправления оценивались по фиксированной таблице решений: включались однострочные исправления, в том числе с косметическими сопутствующими изменениями; исключались многострочные, состоящие из нескольких правок, неоднозначные, рефакторинговые и неалгоритмические. Жёсткий однострочный фильтр — осознанное решение, и его цену следует назвать прямо: он смещает выборку к простым, чисто локализуемым ошибкам и тем самым **сужает разброс, доступный для различения условий**. Это консервативное смещение: оно работает против обнаружения эффекта, а не в его пользу.

Отдельно проверялась работоспособность тестов: для каждой включённой программы фиксировалось, сколько тестов проходит и сколько проваливается на ошибочной версии и что все проходят на исправленной.

2.6 Определение алгоритмических паттернов

Замороженный словарь определяет **двенадцать взаимоисключающих классов**: два указателя, скользящее окно, бинарный поиск, префиксные суммы, динамическое программирование, жадная стратегия, обход графа или сетки (поиск в глубину и в ширину объединены в один класс), «сортировка, затем проход», подсчёт через хеш-таблицу, прямая реализация полным перебором, слияние или планирование интервалов, пошаговая симуляция.

Каждый класс имеет письменное определение и явное правило разрешения спутываемых пар — например, как отличать «два указателя» от «скользящего окна». Словарь опирается на данные о выполнимости такой классификации, рассмотренные в разделе 2.1, и заморожен как для настоящего эксперимента, так и для любого будущего.

2.7 Надёжность разметки

Изначальный протокол предполагал, что паттерны разметит человек-аннотатор. Внутренний аудит, проведённый до получения каких-либо научных результатов, установил, что в предыдущей версии выборки метки фактически назначались жёстко закодированной эвристикой на регулярных выражениях, встроенной в служебный скрипт, без участия человека; прямая проверка шести программ обнаружила две ошибочные классификации — обе в классе «бинарный поиск»: одна метка была вызвана неиспользуемым импортом, другая — случайным совпадением двух не связанных между собой фрагментов кода. Выполнение эксперимента на такой выборке было остановлено.

Полностью протокольная ручная разметка всех 232 программ оказалась невыполнимой в доступное время. Взамен была авторизована и выполнена замена: **независимая слепая разметка двумя моделями**. Аннотатор А (Claude) и аннотатор В (Codex) независимо разметили все 232 программы рамки. Каждый видел только идентификаторы записи и задачи, разрешённые метаданные задачи, ошибочный исходный код и определения классов словаря. Ни один не видел место ошибки, исправленный код, различие между версиями, тексты подсказок, старые эвристические метки и результат другого аннотатора. Сравнение выполнялось только после того, как обе разметки были завершены полностью.

Результат сравнения: сопоставлено 232 программы; **точных совпадений — 182**; расхождений — **50**; необработанное согласие — **78.4483%**; согласие, ожидаемое от случайного совпадения, — 13.00%; **каппа Коэна ≈ 0.752269** .

Что означает каппа Коэна. Простая доля совпадений завышает согласие: два разметчика, случайно выбирающие метки, всё равно иногда совпадут, и тем чаще, чем меньше классов и чем неравномернее они распределены. Каппа Коэна — мера согласия, поправленная на совпадения, ожидаемые по случайности: она равна нулю, если согласие не превышает случайного, и единице при полном совпадении. Здесь случайное согласие составило 13.00%, фактическое — 78.4483%, что даёт $k \approx 0.75$. По общепринятой шкале интерпретации [35] это «значительный» (substantial) уровень — заметно выше случайного, но не идеальный.

Согласие существенно различалось по классам: наивысшее для обхода графа (86.67%) и динамического программирования (84.91%), наинизшее для «двух указателей» (28.57%) и подсчёта через хеш-таблицу (40.00%); полная разбивка приведена в Приложении Г. Согласие также сильно связано с уверенностью аннотаторов: там, где оба указали высокую уверенность, метки совпали в 97.17% случаев.

Это принципиальное ограничение, и оно заявляется прямо: полученные метки являются **точным совпадением двух независимых слепых модельных аннотаций**, а не человеческими и не эталонными. Формулировки «человеческая разметка», «экспертные

метки» или «эталонные метки» в настоящей работе не используются, поскольку были бы неточны. Каппа измеряет *согласованность* двух моделей, а не *правильность* меток: две модели могут разделять общие смещения обучения и ошибаться согласованно.

2.8 Итоговая экспериментальная выборка

Пятьдесят программ, по которым аннотаторы разошлись, было решено **не отдавать на арбитраж третьей модели**, а исключить из эксперимента единообразно, без предпочтения какому-либо из аннотаторов. Причина: арбитраж внёс бы в выборку метки, зависящие от выбора конкретного арбитра, и надёжность разметки перестала бы быть измеримой. Решение принято до получения каких-либо результатов.

Возникающее смещение отбора необходимо назвать прямо. Итоговая выборка состоит только из программ, алгоритмический замысел которых оказался сравнительно однозначным — настолько, что два независимых разметчика назвали его одинаково. Программы с неочевидным, пограничным или смешанным замыслом — а это 21.6% рамки — систематически отсутствуют. Результаты, следовательно, относятся к «чистым» случаям и могут не переноситься на неоднозначные, которые в реальном коде составляют существенную долю. Исключённые 50 программ сохранены в неизменном виде как материал для будущей работы.

После исключения 50 программ расхождения и 25 программ, чей согласованный паттерн лежит вне словаря, осталось 157 программ с однозначной меткой внутри словаря. К ним было применено неизменное, независимое от результата правило: классы ранжируются по числу подходящих программ, отбираются четыре крупнейших при пороге не менее 7 программ, внутри каждого выполняется посеянный случайный отбор без возврата. Итоговый состав выборки приведён в Таблице 1.

Таблица 1 — Итоговый состав экспериментальной выборки (n = 30)

Ранг	Класс алгоритмического паттерна	Доступно программ	Отобрано
1	Динамическое программирование (dynamic_programming)	45	8
2	Прямая реализация полным перебором (brute_force_implementation)	37	8
3	Бинарный поиск (binary_search)	14	7
4	Обход графа в глубину или ширину (graph_traversal_dfs_bfs)	13	7
	Всего	109	30

Примечание. Классы «жадная стратегия» (11 программ) и «пошаговая симуляция» (10) также превысили порог, но заняли пятое и шестое места и потому не отбирались — правило ранжирования было зафиксировано заранее и не пересматривалось. Все 30 программ относятся к 30 различным задачам, имеют длину от 25 до 150 строк и ровно одну ошибочную строку.

После формирования выборка была **зафиксирована окончательно и снабжена контрольной суммой**; ни одно обращение к модели не выполнялось до этой фиксации. Проверка подтвердила: ровно 30 включённых программ, все идентификаторы задач различны, каждая запись имеет точное совпадение двух аннотаторов, ни одна не входит в число 50 программ расхождения, квоты классов соответствуют распределению 8/8/7/7.

2.9 Экспериментальные условия

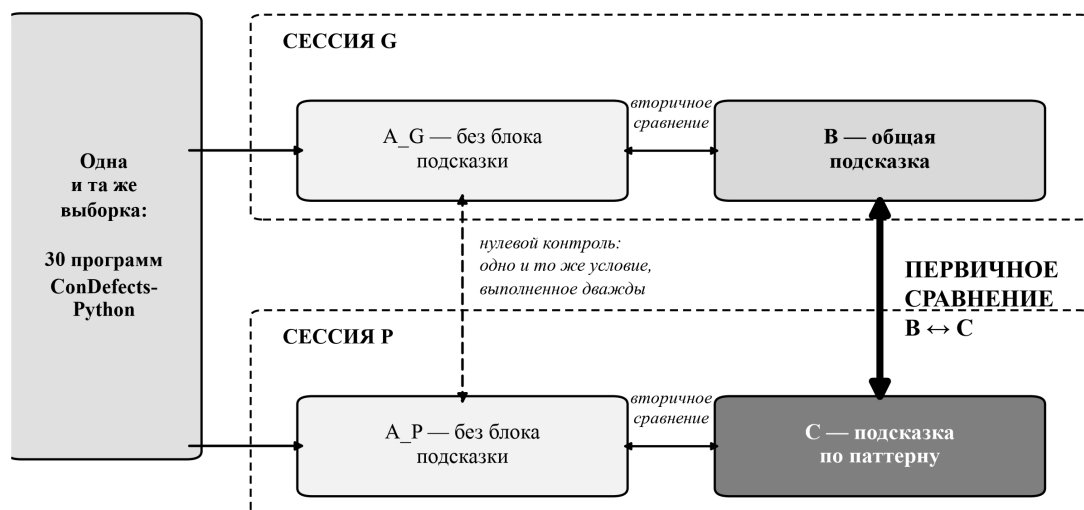
Прежде чем вводить обозначения, поясним замысел простыми словами. Мы хотим проверить, помогает ли модели знание о том, *какой алгоритм* перед ней. Наивный способ — дать модели подсказку про алгоритм и сравнить с тем, что будет без всякой подсказки. Но такой опыт ничего не докажет: вместе с информацией об алгоритме мы добавим сто с лишним лишних слов, нумерованный список и прямое указание «проверь сначала это». Если результат улучшится, мы не узнаем, помогла ли информация об алгоритме или просто любая дополнительная инструкция.

Поэтому вводится **третье, контрольное условие-плацебо**: подсказка того же объёма, того же формата, того же тона и с тем же числом пунктов, но её содержание **не связано** с алгоритмом программы — это общие советы по поиску логических ошибок, одинаковые для всех тридцати программ. Тогда сравнение «подсказка по алгоритму против общей подсказки» изолирует ровно один фактор: выведено ли содержание из паттерна программы. Итоговый набор условий сведён в Таблице 2, а схема их распределения по сессиям — на Рисунке 3.

Таблица 2 — Экспериментальные условия

Обозначение	Сессия	Что вставлено в запрос	Назначение условия
A_G	G	Блок подсказки отсутствует	Внутрисессионный контроль сессии G
B	G	Чек-лист из 4–5 общих пунктов, одинаковый для всех программ	Согласованный контроль-плацебо; база первичного сравнения
A_P	P	Блок подсказки отсутствует	Внутрисессионный контроль сессии P; вместе с A_G образует нулевой контроль
C	P	Чек-лист из 4–5 пунктов, специфичных для класса паттерна данной программы	Проверяемое условие; вторая сторона первичного сравнения

Примечание. **Первичное сравнение** — C против B. Условия A_G и A_P — буквально одно и то же условие, выполненное дважды независимо; их расхождение служит нулевым контролем (раздел 2.12).



5 повторений в каждой ячейке «программа × условие»; всего 600 обращений к модели

Единица анализа — программа: исход Top-K агрегируется по большинству из пяти повторений

Рисунок 3 — Дизайн эксперимента. Первичное сравнение B ↔ C выделено; пунктиром показан нулевой контроль A_G ↔ A_P.

Пример подсказки по паттерну для класса «динамическое программирование»: проверьте, верны ли значение и индекс базового случая; учитывает ли рекуррентное соотношение все предшествующие состояния; согласован ли порядок заполнения таблицы с направлением зависимости; согласована ли инициализация таблицы со способом комбинирования результатов; нет ли рассогласования смещения индексов. Соответствующая ей общая подсказка перечисляет: корректность границ циклов; направление операторов сравнения; инициализацию и порядок обновления накопителя; обработку граничных случаев. Полные тексты приведены в Приложении А.

Согласованность подсказок проверена механически, а не заявлена. По всем двенадцати парам «паттерн — плацебо» число пунктов совпадает точно; расхождение по числу слов не превышает 8 слов при длине подсказок 119–138 слов; вводное предложение,

заключительная оговорка «это гипотезы, а не выводы» и формулировка «проверьте, ...» в каждом пункте идентичны по форме; проверено также, что в тексте плацебо не встречается ни один термин словаря паттернов. Фактически измеренная в эксперименте средняя длина запроса составила 762.53 слова в условии В против 763.10 слова в условии С — расхождение в полслова.

Одна асимметрия сохраняется и раскрывается. Блок подсказки в обоих условиях обернут одним и тем же служебным тегом, название которого содержит слова «алгоритмический паттерн». В условии С этот тег описывает содержание правдиво, в условии В — вводит в заблуждение. Это единственное смещение дизайна, работающее **в пользу** обнаружения эффекта; все остальные выявленные смещения работают против.

Что дизайн не исключает. Общее плацебо одинаково для всех тридцати программ, тогда как подсказка по паттерну выбирается индивидуально для каждой. Поэтому альтернативное объяснение «помогает любая подстройка под конкретную программу, а не именно знание алгоритма» настоящим дизайном не опровергается и остаётся открытым.

Важная оговорка об условии А. Условие «без подсказки» не является условием «без какого-либо руководства». Общая системная инструкция, присутствующая одинаково во всех трёх условиях, уже просит модель рассуждать как внимательный ревьюер кода и обращать внимание на границы циклов, направления сравнений, обновления индексов и инициализацию состояния. Поэтому А означает «без блока подсказки», а не «нулевой уровень помощи», и это раскрывается везде, где приводится сравнение с участием А.

2.10 Модель и параметры эксперимента

Все условия выполнялись **одной замороженной большой языковой моделью** z-ai/glm-5.3-flash, доступной через сервис OpenRouter, с явно закреплённым нижележащим поставщиком вычислений Z.AI и отключёнными резервными поставщиками. Идентичность модели и поставщика проверялась в ответе на каждом обращении: любое несоответствие по протоколу немедленно аннулировало бы весь запуск.

Ключевые параметры: **температура 0** (параметр, управляющий случайностью: при нуле модель должна выбирать наиболее вероятное продолжение), **предельный объём ответа 32 768 токенов**, **усилие внутреннего рассуждения low**, **5 повторений** на каждую ячейку «программа × условие», тайм-аут 120 секунд, 4 параллельных запроса. Всего выполнено **600 научных обращений** (30 программ × 2 сессии × 2 условия × 5 повторений). Полная запись конфигурации приведена в Приложении Д.

Почему эти параметры были заморожены до получения результата. Конфигурация выполнения — инженерное решение, и она действительно уточнялась несколько раз, но **все уточнения происходили до появления хотя бы одного научного результата** и были вызваны одной технической причиной: модель расходовала весь доступный объём ответа на внутреннее рассуждение и не выдавала содержательного ответа вовсе. Последовательно предельный объём повышался с 1024 до 4096, затем до 16 384 и до 32 768 токенов; каждое повышение проверялось синтетическим стресс-тестом на данных, не входящих в выборку. Проблема окончательно устранилась только ограничением усилия рассуждения до уровня low. Все попытки запуска, прерванные до этого момента, были аннулированы целиком и сохранены в проекте как аудиторская история; **ни один их фрагмент не используется как научное свидетельство.**

Существенно, что параметры были зафиксированы **до финального запуска** и не менялись после того, как стали известны какие-либо результаты. Это защищает от подгонки конфигурации под желаемый исход.

Почему пять повторений при температуре 0. Изначально предполагалось, что при температуре 0 модель детерминирована и пять повторений дадут пять одинаковых ответов, то есть будут лишь проверкой этого предположения. Фактически (раздел 2.13) детерминизма не оказалось, и правило агрегации по большинству из пяти повторений выполняло содержательную работу.

2.11 Метрики оценки

Точность Тор-К. Модель возвращает список подозрительных строк, упорядоченный по убыванию подозрительности. Для программы Тор-К равна единице, если истинно ошибочная строка попала в первые K позиций списка, и нулю иначе. Интуитивно: **Тор-1** — модель назвала ошибочную строку первой же; **Тор-3** — ошибочная строка вошла в тройку первых предположений; **Тор-5** — вошла в пятёрку. Тор-1 — самая строгая метрика и выбрана **первичной**: именно она соответствует практической ситуации, когда пользователь проверяет первое предложение инструмента. Тор-3 и Тор-5 — вторичные.

Совпадения по подозрительности разрешаются консервативно: строка, разделяющая оценку с несколькими другими, получает **худшую** позицию внутри этой группы. Случайное попадание за счёт равных оценок не засчитывается в пользу модели.

EXAM*. Метрика Тор-К отвечает «да/нет» и не различает случаи, когда ошибочная строка стоит шестой и когда пятидесятой. EXAM отвечает на другой вопрос: **какую долю файла придётся просмотреть, двигаясь по списку сверху вниз, прежде чем дойти до ошибочной строки.** Пример: если ошибочная строка стоит второй в файле из ста строк, $EXAM = 0.02$; если она вообще не встречается в списке, EXAM принимает максимальное значение 1.0. **Чем меньше EXAM, тем лучше** — это единственная метрика работы, где меньшее значение означает лучший результат.

Звёздочка в обозначении **EXAM*** поставлена намеренно. В качестве знаменателя используется **физическое число строк файла**, а не число содержательно проверяемых операторов, как в оригинальной метрике. Поэтому значения **EXAM*** настоящей работы **численно несопоставимы** с опубликованными значениями EXAM в других работах, и представлять их как сопоставимые нельзя.

Единица анализа — программа, а не обращение к модели. Пять повторений одной ячейки — это повторные измерения одного и того же объекта, а не пять независимых наблюдений. Для каждой программы и каждого условия единый исход Тор-К выводится по правилу большинства, зафиксированному заранее: успех, если попадание произошло хотя бы в 3 из 5 повторений. Значение **EXAM*** усредняется по пяти повторениям. Объединение «30 программ $\times$ 5 повторений» в анализ со 150 наблюдениями **прямо запрещено** планом анализа: это зависило бы точность оценок в несколько раз.

2.12 Статистический анализ

План анализа был зафиксирован до появления данных; ниже изложена его суть в доступной форме.

Точный критерий Макнемара. Это тест для парных наблюдений «да/нет»: одни и те же тридцать программ проверяются в двух условиях. Программы, где оба условия дали одинаковый результат (обе попали или обе промахнулись), для теста **неинформативны**. Работают только **несогласующиеся (дискордантные) пары**: программы, где одно условие попало, а другое промахнулось. Обозначим b — число программ, где попала только подсказка по паттерну; c — где попала только общая подсказка; $d = b + c$ — общее число дискордантных пар. Если ни одно условие не лучше другого, то каждая дискордантная пара с равной вероятностью попадает в b или в c — как подбрасывание монеты. Тест вычисляет, насколько наблюдаемый перекося неправдоподобен при честной монете. Итог — **величина p** : вероятность увидеть расхождение не менее выраженное, чем наблюдаемое, если между условиями разницы нет. Малое p (обычно менее 0.05) означает, что перекося трудно объяснить случайностью; большое p **не означает**, что условия равны, — оно означает лишь, что данные не позволяют их различить.

Ограничение по мощности, известное заранее. При тридцати парах арифметика теста жёстко ограничивает, чего эксперимент вообще способен достичь. Минимально достижимая величина p зависит от числа дискордантных пар: при $d = 3$ это 0.250, при $d = 4$ — 0.125, при $d = 5$ — 0.0625, и только при $d = 6$ она впервые опускается до 0.031. Иными словами, **нужно не менее шести дискордантных пар, причём все в одну сторону, чтобы**

значение $p \leq 0.05$ стало достижимым хотя бы в принципе; при одной паре в противоположную сторону требуется уже девять. Это было вычислено и записано до эксперимента. Отсюда прямое следствие: **нулевой результат при таком объёме выборки неинформативен относительно отсутствия эффекта.**

Доверительный интервал и бутстреп. Величина p говорит только «различно или нет». Доверительный интервал говорит, какие значения истинной разницы **совместимы с данными**. Он вычислялся методом бутстрепа: из тридцати программ 10 000 раз формировалась случайная выборка того же размера с возвращением, для каждой вычислялась разница между условиями, и из полученного распределения брались 2.5-й и 97.5-й процентиля. Зерно генератора случайных чисел зафиксировано, поэтому расчёт воспроизводим в точности.

Проверка нулевым контролем. Это, возможно, наиболее важная защита дизайна. Контрольное условие без подсказки выполнялось дважды — в каждой из двух сессий, на одних и тех же программах, с одними и теми же параметрами. Любое расхождение между A_G и A_P объясняется **исключительно** повторным запуском: недетерминизмом модели и дрейфом на стороне поставщика. Это даёт эмпирический «пол шума». Заранее было записано правило: если число дискордантных пар в нулевом контроле окажется **не меньше**, чем в первичном сравнении, первичное сравнение объявляется неинтерпретируемым при данном объёме выборки — независимо от того, какое значение p оно даст. Правило зафиксировано до того, как стала известна хотя бы одна из двух величин.

Множественность и дисциплина отчётности. Первичной назначена ровно **одна** конечная точка: $Top-1$, C против B . Все остальные величины — вторичные или описательные, сообщаются без поправки на множественность и **не дают права на вывод**. Защита от избирательной отчётности состоит не в статистической поправке, а в том, что полный список сообщаемых величин был зафиксирован заранее, и каждая обязана быть сообщена независимо от того, благоприятна ли она для гипотезы. Дополнительно планом предусмотрены: критерий знаковых рангов Уилкоксона для попрограммных разностей EXAM*, описательная разбивка по классам паттернов без инференциальных тестов и проверка устойчивости по каждому из пяти повторений в отдельности.

2.13 Результаты

Результаты излагаются в порядке, зафиксированном планом анализа до сбора данных: целостность выполнения → нулевой контроль → первичное сравнение → вторичные показатели → описательная разбивка.

Целостность выполнения. Обе сессии выполнены полностью, в зафиксированном порядке, с жёсткой проверкой согласованности между ними. Выполнено **600 обращений к модели из 600 запланированных**: по 300 в каждой сессии. Отказов на стороне поставщика — **0**. Отказов разбора ответа — **1** (одно обращение из 150 в условии B , 0.67%); по заранее зафиксированной политике оно засчитано как промах, а не исключено из анализа: исключение было бы более выгодным для результата и потому запрещено протоколом. Все проверки согласованности между сессиями пройдены: совпадение контрольных сумм выборки, системной инструкции, шаблона сообщения и запросов по каждой программе; идентичность поставщика, модели, температуры, лимита токенов, усилия рассуждения и числа повторений.

Незапланированный, но важный результат: детерминизма при температуре 0 не оказалось. Доля ячеек «программа × условие», где все пять ответов совпали побайтово, составила **0.00% во всех четырёх условиях**. Доля ячеек, где совпали все пять исходов $Top-1$, составила 56.67% (A_G), 73.33% (A_P), 66.67% (B) и 66.67% (C). Практический смысл наблюдения выходит за рамки настоящей работы: **однократный запрос к подобной модели в задаче локализации не воспроизводим**, и любой протокол оценивания обязан включать повторения и явное правило агрегации.

Проверка нулевым контролем. Одно и то же контрольное условие, выполненное дважды независимо, дало $Top-1$ **13/30 (43.33%)** в сессии G и **12/30 (40.00%)** в сессии P .

Расхождение составило -3.33 п.п. при **пяти** дискордантных программах; точный критерий Макнемара дал $p = 1.0$. Поскольку число дискордантных пар нулевого контроля (5) оказалось **меньше** числа дискордантных пар первичного сравнения (6), заранее заявленная проверка **пройдена**, и первичное сравнение признаётся интерпретируемым при данном объёме выборки.

Эта проверка отвечает на вопрос, породили ли две отдельные сессии сами по себе существенный дрейф, не связанный с манипуляцией подсказкой. Ответ: породили, и **запас минимален**. Простой повторный запуск буквально одного и того же условия дал расхождение по пяти программам из тридцати; первичное сравнение — с двумя содержательно разными подсказками — по шести. **Уровень шума в этом эксперименте почти равен всему объёму сигнала, который сравнение способно уловить**. Эта величина — эмпирическая оценка «шумового пола» — имеет самостоятельную ценность для планирования будущих исследований.

Первичный результат: гипотеза H1 не подтверждена. Общая подсказка (B) и подсказка по паттерну (C) дали **одинаковую** точность Top-1: **12 из 30 программ (40.00%)** в обоих условиях. Разница — **0.00 процентного пункта**. Точный критерий Макнемара дал $p = 1.0$ при шести дискордантных программах, распределённых ровно поровну: **b = 3** программы локализованы только при подсказке по паттерну, **c = 3** — только при общей подсказке. Бутстреп-интервал 95% для разницы C – B составил **$[-16.67; +16.67]$ п.п.**

Как это читать — и как не читать. Значение $p = 1.0$ особенно легко истолковать неверно, поэтому формулировка должна быть точной. Оно **не означает**, что два условия эквивалентны; **не означает**, что подсказка по паттерну бесполезна; **не означает**, что эффекта не существует. Оно означает ровно одно: среди программ, по которым условия разошлись, расхождение оказалось симметричным, и наблюдаемая картина настолько согласуется с нулевой гипотезой, насколько это вообще возможно. Доверительный интервал делает границы знания явными: данные совместимы с истинным **ухудшением почти на 17 п.п.** и с истинным **улучшением почти на 17 п.п.** Формальная проверка эквивалентности планом анализа не предусматривалась, не проводилась и не может быть подставлена задним числом. Дополнительно: план заранее установил, что для достижимости $p \leq 0.05$ требуется не менее шести **однонаправленных** дискордантных пар. Наблюдалось ровно шесть — но направленных поровну. Эксперимент, таким образом, работал в точности на границе своей разрешающей способности.

Вторичные показатели. Все приведённые ниже величины — **вторичные или описательные**. Они сообщаются без поправки на множественность и, согласно замороженному плану анализа, **не дают права ни на какой вывод**. Ни один из них не достигает традиционного порога значимости, и каждый доверительный интервал включает ноль. Сводка приведена в Таблице 3 и на Рисунках 4 и 5.

Таблица 3 — Основные результаты: общая подсказка (В) против подсказки по паттерну (С), n = 30 программ

Показатель	В	С	Разница (С – В)	95% ДИ	Дискордантные пары	Критерий	p	Статус
Тор-1	12/30 (40.00%)	12/30 (40.00%)	0.00 п.п.	[–16.67; +16.67] п.п.	$b = 3, c = 3, d = 6$	Точный Макнемара	1.0	Первичный
Тор-3	15/30 (50.00%)	18/30 (60.00%)	+10.00 п.п.	[–6.67; +26.67] п.п.	$b = 5, c = 2, d = 7$	Точный Макнемара	.453	Вторичный
Тор-5	16/30 (53.33%)	19/30 (63.33%)	+10.00 п.п.	[–3.33; +23.33] п.п.	$b = 4, c = 1, d = 5$	Точный Макнемара	.375	Вторичный
ЕХАМ* (меньше — лучше)	0.510798	0.445055	–0.065743	[–0.1622; +0.0283]	—	Уилкоксон	.124	Вторичный

Примечание. Доверительный интервал — кластеризованный по программам бутстреп, 10 000 итераций, зафиксированное зерно. b — число программ, локализованных только при подсказке по паттерну; c — только при общей подсказке. Для ЕХАМ* отрицательная разница означает улучшение. Для справки: контрольные условия без подсказки дали Тор-1 43.33% (A_G) и 40.00% (A_P); полные показатели всех четырёх условий — в Приложении В.

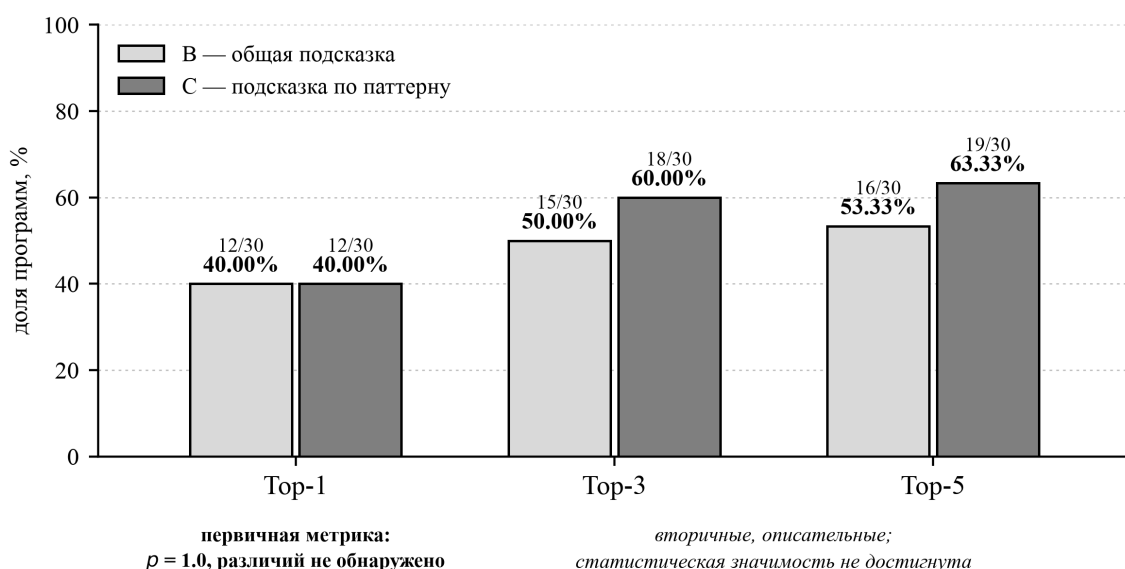


Рисунок 4 — Точность локализации ошибки по метрикам Тор-К. Значения над столбцами — доля программ, для которых истинно ошибочная строка попала в первые К позиций.

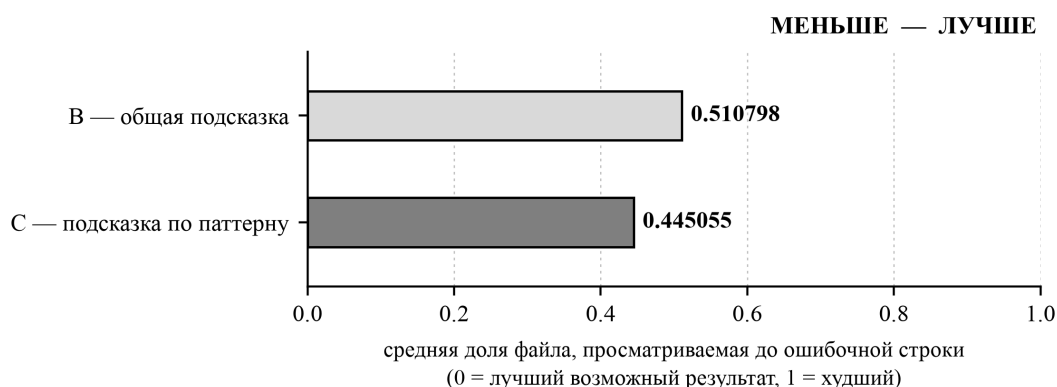


Рисунок 5 — Сравнение EXAM*. Различие описательное, статистическая значимость не достигнута ($p = .124$). EXAM* вычисляется по физическому числу строк файла и не сопоставим с опубликованными значениями EXAM в других работах.

Вторичные показатели **последовательно направлены в пользу условия С**: подсказка по паттерну сопровождается на 10.00 п.п. более высокой точностью и в Top-3, и в Top-5, а также меньшей (то есть лучшей) средней долей просматриваемого файла. Однако каждый доверительный интервал включает ноль, а величины p (.453, .375 и .124) далеки от традиционного порога. Согласованность направления по трём показателям представляет интерес как **гипотеза**, но не как установленный факт, и по двум причинам сразу: при данном объёме выборки такое совпадение направлений неотличимо от случайности, а сами три показателя не независимы — Top-3, Top-5 и EXAM* вычисляются по одному и тому же списку.

Сравнения с условием без подсказки. Ни одно из условий с подсказкой не превзошло условие без блока подсказки по первичной метрике: переход $A_G \rightarrow B$ дал 43.33% против 40.00% ($p = 1.0$), переход $A_P \rightarrow C$ дал 40.00% против 40.00% ($p = 1.0$). Вторичное исследовательское ожидание, согласно которому обе подсказки должны превзойти контроль, **не подтвердилось**. Эти сравнения обязаны сопровождаться напоминанием, что условие A не является условием «без какого-либо руководства» (раздел 2.9).

Диагностика. Различия во вторичных показателях не могут объясняться объёмом текста: средняя длина запроса составила 762.53 слова (B) против 763.10 (C), а средняя длина возвращённого списка — 3.02 против 3.08 элемента. Ни «больше текста в подсказке», ни «длиннее список кандидатов» наблюдаемого сдвига не объясняют.

Описательные результаты по классам паттернов. Разбивка по классам (Таблица 4, Рисунок 6) обнаруживает структуру, которую объединённый результат полностью скрывает.

Таблица 4 — Результаты по алгоритмическим классам (описательные величины)

Класс паттерна	n	Top-1 (B)	Top-1 (C)	Δ Top-1 (C – B)	EXAM* (B)	EXAM* (C)
Полный перебор (brute_force_implementation)	8	5/8 (62.50%)	7/8 (87.50%)	+25.00 п.п.	0.269640	0.170171
Динамическое программирование (dynamic_programming)	8	4/8 (50.00%)	2/8 (25.00%)	–25.00 п.п.	0.374119	0.419406
Бинарный поиск (binary_search)	7	1/7 (14.29%)	1/7 (14.29%)	0.00 п.п.	0.838505	0.752966
Обход графа (graph_traversal_dfs_bfs)	7	2/7 (28.57%)	2/7 (28.57%)	0.00 п.п.	0.614905	0.480609

Примечание. Исключительно описательно; не предназначено для подтверждающего вывода. Ни один инференциальный тест на уровне класса не выполнялся и не сообщается: 7–8 программ на класс их не поддерживают. Сдвиг на 25 п.п. в классе из восьми программ соответствует разнице **всего в две программы**.

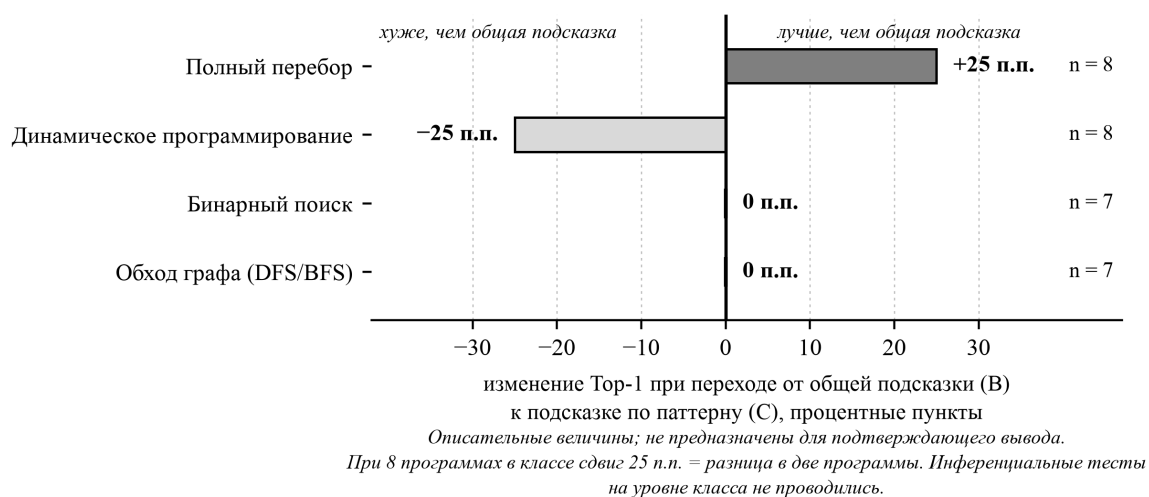


Рисунок 6 — Изменение Топ-1 по алгоритмическим классам. Описательные результаты; не предназначены для подтверждающего вывода.

Ключевое наблюдение: **направление эффекта различалось по классам алгоритмов**. В классе прямого полного перебора подсказка по паттерну сопровождалась выигрышем в 25 п.п. (7 из 8 против 5 из 8), в классе динамического программирования — проигрышем в 25 п.п. (2 из 8 против 4 из 8). Два ненулевых сдвига равны по модулю, противоположны по знаку и **в точности взаимно погашаются** в объединённом результате Топ-1, давая наблюдаемый ноль. Осторожность здесь обязательна дважды: каждый сдвиг — это разница в две программы из восьми, а равенство модулей вполне может быть совпадением.

2.14 Анализ результатов

Три рамочных положения верны безусловно, поскольку следуют из дизайна, а не из данных, и должны предшествовать любой интерпретации. Первое: только сравнение С против В говорит о специфичности паттерна; различия между условиями с подсказкой и без неё измеряют эффект получения *какой-либо* инструкции. Второе: это калибровочный эксперимент при $n = 30$, и никакой его исход не даёт права заявить, что гипотеза доказана или опровергнута. Третье: метки паттернов получены точным совпадением двух независимых слепых модельных аннотаций, а 50 программ расхождения исключены, а не разрешены арбитражем.

Уровень 1 — что известно. Специфичная для алгоритмического паттерна структурная подсказка **не улучшила строгую локализацию Топ-1** по сравнению с согласованной общей подсказкой. Наблюдаемая разница — ровно ноль, при шести дискордантных программах, разделившихся поровну. Из этого **не следует**, что информация об алгоритмическом паттерне бесполезна: доверительный интервал $[-16.67; +16.67]$ п.п. совместим с существенным эффектом любого знака, и при тридцати программах эксперимент попросту не обладает разрешающей способностью, чтобы отличить заметный эффект от его отсутствия. Нулевой исход при таком объёме выборки был назван наиболее вероятным ещё в замороженном плане анализа.

Существенно и то, что нулевой результат получен **несмотря на** единственную асимметрию дизайна, работавшую в пользу гипотезы. Все прочие выявленные смещения — однострочный фильтр ошибок, отбор только по согласию аннотаторов, перекрытие общего чек-листа с неспецифичными частями чек-листов по паттернам — работают против обнаружения эффекта. Измеренная разница $C - B$, таким образом, является скорее консервативной нижней границей, чем завышенной оценкой.

Уровень 2 — на что указывают вторичные показатели. Наиболее содержательное наблюдение эксперимента — не сам ноль, а **расхождение между строгой метрикой первого выбора и метриками качества ранжирования**. Подсказка по паттерну не изменила долю программ, где ошибочная строка заняла первое место (40.00% против 40.00%), но

сопровождалась более высокими Топ-3 (60.00% против 50.00%) и Топ-5 (63.33% против 53.33%) и меньшей средней долей просматриваемого файла (0.445055 против 0.510798). Все три показателя направлены в одну сторону.

Возможное объяснение: структурная информация о паттерне влияет не столько на то, какую строку модель называет **первой**, сколько на то, **куда она помещает правильный ответ, когда не ставит его первым**, — то есть подтягивает ошибочную строку в верхнюю часть списка, не меняя его вершину. Механизм согласуется с диагностикой: длина списков практически одинакова (3.02 против 3.08 элемента), а запросы различаются на полслова, поэтому ни «более длинный ответ», ни «более объёмная подсказка» сдвига не объясняют.

Эту интерпретацию следует читать строго как гипотезу для будущего исследования. Ни один из трёх показателей не достигает значимости, каждый интервал включает ноль, все три вторичны и взаимозависимы. Перенос вывода на вторичные показатели после того, как первичный оказался нулевым, был бы именно той избирательной отчётностью, против которой план анализа фиксировался заранее. Правильный статус наблюдения: **кандидат в первичные конечные точки следующего исследования**, а не результат настоящего.

Уровень 3 — что остаётся неизвестным. Контраст между классами — +25 п.п. для полного перебора против -25 п.п. для динамического программирования — наиболее интригующий и требует наибольшей осторожности. Правдоподобные механизмы можно назвать, но каждый следует явно пометить как *возможное объяснение*, а не установленную причину.

Возможное объяснение 1 — различная информативность чек-листа. Для программ полного перебора пространство ошибок узко и почти исчерпывается содержанием подсказки: границы диапазона, точность следования формуле, порядок операций, однократность финального шага. Структурная подсказка здесь фактически перечисляет большинство мест, где реально может быть ошибка, и продуктивно сужает поиск. Для динамического программирования пространство ошибок шире, а чек-лист — базовый случай, полнота переходов, порядок заполнения, инициализация таблицы — перечисляет *типичные* места сбоя, которые в конкретной программе могут оказаться корректными.

Возможное объяснение 2 — отвлечение внимания. Если перечисленные в подсказке места в данной программе исправны, а ошибка находится в другом месте, подробный чек-лист может систематически **уводить внимание модели** от настоящей ошибки к правдоподобным, но в данном случае верным фрагментам. Механизм «подсказка, направляющая внимание не туда» — именно тот риск, который стоит проверять целенаправленно, и его существование нельзя исключить по данным настоящего эксперимента.

Возможное объяснение 3 — совпадение. При восьми программах в классе сдвиг на 25 п.п. — это две программы. Равенство модулей двух сдвигов и их точное взаимное погашение выглядят выразительно, но при таких числах вполне могут быть случайностью. Эту возможность нельзя ставить ниже двух предыдущих: она объясняет данные ничуть не хуже.

Данные настоящего эксперимента не позволяют выбрать между тремя версиями. Прямое следствие для будущей работы — стратификация по классам и достаточное число программ внутри каждого класса, а не подбор объяснения задним числом. Общий вывод третьего уровня формулируется осторожно: **структурная информация о паттерне может быть полезна для одних алгоритмических классов и бесполезна или вредна для других**, и это гипотеза, а не установленный факт.

Почему это результат, а не неудача. Во-первых, конвейер отработал от начала до конца на реальных данных: 600 обращений из 600, нулевой уровень отказов поставщика, подтверждённые проверки согласованности между сессиями. Во-вторых, измерен эмпирический шумовой пол: повторный запуск буквально одного и того же условия дал расхождение по пяти программам из тридцати — почти столько же, сколько дало всё первичное сравнение. В-третьих, задокументировано отсутствие детерминизма при

температуре 0, что методологически значимо далеко за пределами настоящей темы. В-четвёртых, и это следует раскрыть прямо: **одну из своих заявленных задач эксперимент выполнить не смог**. Калибровочное исследование должно было оценить частоту расхождения между условиями, чтобы по ней рассчитать необходимый объём будущего подтверждающего исследования. Частота расхождения получена (6 из 30, то есть 0.200, с бутстреп-интервалом [0.067; 0.367]), но коэффициент направления оказался в точности равен 0.5 — три программы в одну сторону, три в другую. При этом значении стандартная формула расчёта объёма выборки обращается в неопределённость: её знаменатель равен нулю. Зарегистрированный статус расчёта — «эксперимент не смог определить объём исследования».

2.15 Ограничения исследования

Ограничения ниже — не оговорки и не извинения. Это **границы, внутри которых полученный результат допускает истолкование**; за их пределами он не говорит ничего.

- **Объём выборки и статистическая мощность.** При 30 программах точный критерий Макнемара не может достичь традиционной значимости, если менее шести дискордантных пар не направлены в одну сторону. Наблюдалось ровно шесть, но поровну.
- **Широкий доверительный интервал.** Интервал $[-16.67; +16.67]$ п.п. совместим с существенным эффектом любого знака.
- **Нулевой результат — не проверка эквивалентности.** Процедура equivalence testing планом не предусматривалась и не проводилась. Значение $p = 1.0$ отражает симметричность расхождений, а не установленное отсутствие различий.
- **Шумовой пол сопоставим с объёмом сигнала.** Повторный запуск идентичного условия дал расхождение по пяти программам из тридцати. Заявленная проверка формально пройдена, но с минимальным запасом.
- **Отсутствие детерминизма при температуре 0.** Ни в одной ячейке пять повторений не дали побайтово идентичных ответов. Это добавляет шум во все условия одинаково — то есть не создаёт систематического смещения в чью-либо пользу, — но снижает точность всех оценок.
- **Одна модель.** Результат специфичен для одной названной модели при одном закреплённом маршруте поставщика, температуре 0 и **вынужденно ограниченном усилии внутреннего рассуждения**. Утверждение вида «большие языковые модели не выигрывают от подсказок по паттерну» одномодельным дизайном не поддерживается. Нельзя исключить, что при более высоком усилии рассуждения взаимодействие модели со структурной подсказкой было бы иным.
- **Один язык программирования.** Все 30 программ написаны на Python; перенос на Java, C++ и другие языки предполагать нельзя.
- **Один набор данных.** ConDefects собран исключительно из соревновательных решений, которые систематически отличаются от учебных заданий и профессионального кода и по стилю, и по распределению ошибок.
- **Только однострочные ошибки.** Ограничение однозначными, чисто локализуемыми ошибками устраняет более трудные многострочные и рефакторинговые случаи — вероятно, именно те, где алгоритмическая структура проявляется сильнее всего.
- **Смещение отбора «только по согласию».** Итоговая выборка содержит только программы, по которым два независимых аннотатора дали точно совпадающие метки (182 из 232). Программы с неоднозначным алгоритмическим замыслом — 21.6% рамки — систематически отсутствуют.
- **Исключение 50 программ расхождения.** Они не подвергались арбитражу и не были разрешены, а исключены. Решение осознанное и специфичное для калибровочного этапа, но оно ограничивает обобщаемость.

- **Аннотаторы — модели, а не эксперты-люди.** Измеренное согласие ($\kappa \approx 0.75$) соответствует «значительному» уровню, но две модели могут разделять общие смещения обучающих данных и ошибаться согласованно способом, отличным от того, как коррелируют ошибки двух независимо подготовленных людей. **Согласие между моделями — статистика надёжности, а не доказательство правильности меток.**
- **Определение места ошибки выполнялось одним конвейером.** Вторая независимая оценка надёжности на этом этапе не проводилась — допустимое для калибровочного этапа послабление, раскрываемое явно.
- **Отсутствие классических базовых методов в самом эксперименте.** Спектральная и мутационная локализация не выполнялись на этих же тридцати программах; соответствующие значения цитируются из опубликованной работы на другой выборке и служат контекстом, а не сравнением.
- **Возможная контаминация набора данных.** ConDefects собран из общедоступных решений. Степень пересечения обучающих данных модели с этим корпусом не проверялась, и её нельзя исключить.
- **Провенанс тестовых данных.** Упакованные пары «вход — выход» происходят из воспроизводимого стороннего зеркального архива, а не из подтверждённого официального источника; на задачу приходится от 3 до 8 тестовых случаев (211 всего).
- **Исключение условия задачи.** Ни один запрос не содержал формулировки задачи. Это предотвращает утечку информации о паттерне в контрольные условия, но представляет собой намеренное упрощение по сравнению с реальным развёртыванием.
- **Ограничения словаря паттернов.** Двенадцать классов объединяют поиск в глубину и в ширину в один класс и исключают программы, чей метод лежит вне словаря; четыре отобранных класса — дальнейшее сужение.
- **Одна асимметрия в пользу гипотезы.** Служебный тег правдиво называет содержание в условии С и вводит в заблуждение в условии В. Это единственное смещение, работающее в пользу обнаружения эффекта.
- **Отсутствие исследования с участием людей.** Ни один студент, преподаватель или человек-отладчик в эксперименте не участвовал. Никакое утверждение о скорости отладки, понимании или образовательном результате настоящей работой не поддерживается; результаты **не доказывают** никакого учебного эффекта платформы AlgoRythm.

2.16 Практическое значение для AlgoRythm

AlgoRythm — учебная платформа, сочетающая разбор алгоритмических паттернов с песочницей для написания кода. Настоящее исследование выполнено, чтобы ответить на конкретный проектный вопрос: имеет ли смысл строить в платформе отладочного помощника, обусловленного распознанным алгоритмическим паттерном.

Ответ, который даёт эксперимент, конкретен: **оснований внедрять такого помощника как обоснованное улучшение настоящий эксперимент не даёт.** По строгой метрике первого выбора преимущества нет; по остальным метрикам вопрос остаётся неопределённым. Это практически полезный результат: он предотвращает внедрение недоказанной функции под видом обоснованного решения. Вопрос при этом остаётся **открытым, а не решённым отрицательно**, и следующий шаг — исследование достаточной мощности, а не разработка функции.

Потенциальное применение, которое результат не опровергает и которое стоит проверять дальше: платформа могла бы использовать информацию об алгоритмической структуре задания для проектирования более адресной отладочной поддержки — в частности, разной для разных семейств алгоритмов, как подсказывает разнонаправленность эффекта по классам (раздел 2.14).

Границу необходимо повторить: **никакого улучшения обучения, никакого образовательного эффекта, никакого влияния на оценки или результаты студентов** настоящая работа не показывает и не может показывать. Исследование не включало ни одного человека-испытуемого.

2.17 Направления будущих исследований

1. **Подтверждающее исследование достаточного объёма.** Главный дефицит настоящей работы — мощность. Объём **не может быть рассчитан по её данным** (коэффициент направления оказался равен 0.5, и формула вырождается); планировать придётся от содержательно заданной минимальной значимой разницы, используя оценку частоты расхождения 0.200 [0.067; 0.367] лишь как вспомогательную величину.
2. **Арбитраж 50 программ расхождения** третьим независимым аннотатором — чтобы либо включить их в расширенную выборку, либо подтвердить обоснованность исключения и оценить величину смещения отбора.
3. **Разметка экспертами-людьми** на расширенном корпусе, устраняющая зависимость меток от согласия между моделями и позволяющая измерить, насколько модельный консенсус совпадает с экспертным суждением.
4. **Несколько языковых моделей** разных поставщиков, размеров и уровней усилия рассуждения. Одномодельный результат не переносится.
5. **Прямое сравнение со спектральной и мутационной локализацией на той же выборке**, а не цитирование опубликованных значений с другой выборки.
6. **Конвейер с предсказанной, а не консенсусной меткой.** Реалистичное развёртывание требует обученного классификатора, который ошибается. Измерение того, какая доля эффекта сохраняется при автоматической, подверженной ошибкам разметке, — сквозная оценка всей цепочки «классификатор → подсказка → локализация».
7. **Дополнительные языки программирования и наборы данных** помимо ConDefects.
8. **Многострочные и более трудные ошибки**, снятие однострочного фильтра — вероятно, именно там алгоритмическая структура проявляется сильнее всего.
9. **Исследование с участием людей**, сравнивающее скорость и точность отладки новичками с доступом к структурной подсказке и без него. Проводится только после получения необходимого предварительного одобрения.
10. **Проверка гипотезы о разной форме подсказки для разных классов.** Наблюдение раздела 2.13 прямо предполагает, что один и тот же формат чек-листа может быть удачным для одних алгоритмов и вредным для других.

Отдельно: две гипотезы, порождённые настоящей работой — о влиянии на качество ранжирования, а не на первый выбор, и о зависимости эффекта от класса алгоритма, — должны проверяться с **заранее назначенной** соответствующей первичной конечной точкой, а не подтверждаться задним числом по вторичным показателям.

3. ЗАКЛЮЧЕНИЕ

Что изучалось. Улучшает ли структурная подсказка, выведенная из алгоритмического паттерна программы, автоматическую локализацию логической ошибки по сравнению с согласованной по формату, длине и тону общей отладочной подсказкой.

Как это проверялось. Построена воспроизводимая выборка из 30 программ на Python с ровно одной подтверждённой ошибочной строкой, отобранных посеянным конвейером из корпуса ConDefects-Python по четырём классам алгоритмических паттернов. Метки паттернов получены точным совпадением двух независимых слепых модельных аннотаций (согласие 78.4483%, κ Коэна ≈ 0.752269 по 232 программам рамки). Разработана пара подсказок — специфичная для паттерна и общая, — механически согласованных по объёму, формату и числу пунктов. Протокол, параметры модели и план анализа зафиксированы до сбора данных; выполнены все 600 запланированных обращений к одной замороженной модели при температуре 0 с пятью повторениями.

Первичный результат. Гипотеза H1 **не подтверждена**. Точность Top-1 составила 12 из 30 (40.00%) в обоих сравниваемых условиях; разница 0.00 п.п., точный критерий Макнемара $p = 1.0$, 95% доверительный интервал $[-16.67; +16.67]$ п.п.

Вторичный сигнал. Описательные показатели направлены в пользу подсказки по паттерну — Top-3 60.00% против 50.00%, Top-5 63.33% против 53.33%, EXAM* 0.445055 против 0.510798, — но ни один не достигает статистической значимости и каждый доверительный интервал включает ноль. Направление эффекта различалось по классам алгоритмов: +25 п.п. для полного перебора и –25 п.п. для динамического программирования, с точным взаимным погашением в объединённом результате.

Что можно заключить. Специфичная для алгоритмического паттерна структурная информация **не повысила строгую точность локализации Top-1** в настоящем калибровочном эксперименте. Этот результат **не является свидетельством эквивалентности условий и не является свидетельством отсутствия эффекта**: доверительный интервал совместим с существенным эффектом любого знака, а измеренный шумовой пол повторного запуска (пять расходящихся программ из тридцати) почти равен всему объёму расхождения первичного сравнения. Вместе с тем вторичные результаты Top-3, Top-5 и EXAM* допускают, что структурная информация может влиять на **качество ранжирования**, а направление эффекта — зависеть от класса алгоритма.

Что следует проверить далее. Требуется подтверждающее исследование большего объёма с заранее назначенной первичной конечной точкой, стратификацией по алгоритмическим классам, несколькими моделями и арбитражем исключённых программ расхождения. Для платформы AlgoRythm вывод однозначен: внедрять отладочного помощника, обусловленного паттерном, как обоснованное улучшение пока нет оснований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. du Boulay B. Some difficulties of learning to program. *Journal of Educational Computing Research*. 1986; 2(1): 57–73. DOI: 10.2190/3LFX-9RRF-67T8-UVK9.
2. Fitzgerald S., Lewandowski G., McCauley R., Murphy L., Simon B., Thomas L., Zander C. Debugging: Finding, fixing and flailing, a multi-institutional study of novice debuggers. *Computer Science Education*. 2008; 18(2): 93–116. DOI: 10.1080/08993400802114508.
3. Spohrer J.C., Soloway E. Novice mistakes: Are the folk wisdoms correct? *Communications of the ACM*. 1986; 29(7): 624–632. DOI: 10.1145/6138.6145.
4. Alzahrani N., Vahid F. Common logic errors for programming learners: A three-decade literature survey. In: 2021 ASEE Annual Conference & Exposition. American Society for Engineering Education; 2021. DOI: 10.18260/1-2--36814.
5. Yang Y., Mei A., Yang C. An empirical study of MBFL on novice programs across different programming languages. *International Journal of Software Engineering and Knowledge Engineering*. 2025; 35(7): 1037–1070. DOI: 10.1142/S0218194025500329.
6. Xu H., Liu H., Wu Y., Kang X., Chen X., Liu Y. Exploring the potential and limitations of large language models for novice program fault localization. *arXiv preprint*. 2025. URL: <https://arxiv.org/abs/2512.03421>.
7. Watanobe Y., Rahman M.M., Amin M.F., Kabir R. Identifying algorithm in program code based on structural features using CNN classification model. *Applied Intelligence*. 2023; 53(10): 12210–12236. DOI: 10.1007/s10489-022-04078-y.
8. Brown J.S., Burton R.R. Diagnostic models for procedural bugs in basic mathematical skills. *Cognitive Science*. 1978; 2(2): 155–192. DOI: 10.1207/s15516709cog0202_4.
9. Jones J.A., Harrold M.J., Stasko J. Visualization of test information to assist fault localization. In: *Proceedings of the 24th International Conference on Software Engineering*. New York: Association for Computing Machinery; 2002. p. 467–477.
10. Wong W.E., Debroy V., Gao R., Li Y. The DStar method for effective software fault localization. *IEEE Transactions on Reliability*. 2014; 63(1): 290–308. DOI: 10.1109/TR.2013.2285319.
11. Wong W.E., Gao R., Li Y., Abreu R., Wotawa F. A survey on software fault localization. *IEEE Transactions on Software Engineering*. 2016; 42(8): 707–740. DOI: 10.1109/TSE.2016.2521368.
12. Araújo E.P.O., Gaudencio M., Serey D., Figueiredo J. Applying spectrum-based fault localization on novice's programs. In: 2016 IEEE Frontiers in Education Conference (FIE). IEEE; 2016. DOI: 10.1109/FIE.2016.7757727.
13. DeMillo R.A., Lipton R.J., Sayward F.G. Hints on test data selection: Help for the practicing programmer. *Computer*. 1978; 11(4): 34–41. DOI: 10.1109/C-M.1978.218136.
14. Hu Y., Ahmed U.Z., Mehtaev S., Leong B., Roychoudhury A. Re-factoring based program repair applied to programming assignments. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE; 2019. p. 388–398. DOI: 10.1109/ASE.2019.00044.
15. Li Z., Wu S., Liu Y., Shen J., Wu Y., Zhang Z., Chen X. VsusFL: Variable-suspiciousness-based fault localization for novice programs. *Journal of Systems and Software*. 2023; 205: Article 111822. DOI: 10.1016/j.jss.2023.111822.
16. Gupta R., Kanade A., Shevade S. Neural attribution for semantic bug-localization in student programs. In: *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. 2019.
17. MacNeil S., Denny P., Tran A., Leinonen J., Bernstein S., Hellas A., Sarsa S., Kim J. Decoding logic errors: A comparative study on bug detection by students and large language models. In: *Proceedings of the 26th Australasian Computing Education Conference*. New York: Association for Computing Machinery; 2024. p. 11–20. DOI: 10.1145/3636243.3636245.
18. Hoq M., Rao A., Jaishankar R., Piryani K., Janapati N., Vandenberg J., Mott B., Norouzi N., Lester J., Akram B. Automated identification of logical errors in programs: Advancing scalable analysis of student

- misconceptions. In: Proceedings of the 18th International Conference on Educational Data Mining (EDM 2025). International Educational Data Mining Society; 2025. DOI: 10.5281/zenodo.15870203.
19. Neumüller D., Sihler F., Straub R., Tichy M. Exploring the effectiveness of abstract syntax tree patterns for algorithm recognition. In: 2024 IEEE International Conference on Code Quality (ICCQ). IEEE; 2024. DOI: 10.1109/ICCQ60895.2024.10576984.
 20. Brooks R. Towards a theory of the comprehension of computer programs. *International Journal of Man-Machine Studies*. 1983; 18(6): 543–554. DOI: 10.1016/S0020-7373(83)80031-5.
 21. Pennington N. Stimulus structures and mental representations in expert comprehension of computer programs. *Cognitive Psychology*. 1987; 19(3): 295–341. DOI: 10.1016/0010-0285(87)90007-7.
 22. von Mayrhauser A., Vans A.M. Program comprehension during software maintenance and evolution. *Computer*. 1995; 28(8): 44–55. DOI: 10.1109/2.402076.
 23. Baron N., Feitelson D.G. Why is recursion hard to comprehend? An experiment with experienced programmers in Python. In: Proceedings of the 2024 Conference on Innovation and Technology in Computer Science Education. Vol. 1. New York: Association for Computing Machinery; 2024. DOI: 10.1145/3649217.3653636.
 24. du Boulay B., O’Shea T., Monk J. The black box inside the glass box: Presenting computing concepts to novices. *International Journal of Man-Machine Studies*. 1981; 14(3): 237–249. DOI: 10.1016/S0020-7373(81)80056-9.
 25. Sorva J. Notional machines and introductory programming education. *ACM Transactions on Computing Education*. 2013; 13(2): Article 8. DOI: 10.1145/2483710.2483713.
 26. Fincher S., Jeuring J., Miller C.S., Donaldson P., du Boulay B., Hauswirth M., Hellas A., Hermans F., Lewis C., Mühling A., Pearce J.L., Petersen A. Notional machines in computing education: The education of attention. In: Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education. New York: Association for Computing Machinery; 2020. p. 21–50. DOI: 10.1145/3437800.3439202.
 27. Johnson W.L. Understanding and debugging novice programs. *Artificial Intelligence*. 1990; 42(1): 51–97. DOI: 10.1016/0004-3702(90)90094-G.
 28. Ko A.J., Myers B.A. Debugging reinvented: Asking and answering why and why not questions about program behavior. In: Proceedings of the 30th International Conference on Software Engineering. New York: Association for Computing Machinery; 2008. p. 301–310. DOI: 10.1145/1368088.1368130.
 29. Altadmri A., Brown N.C.C. 37 million compilations: Investigating novice programming mistakes in large-scale student data. In: Proceedings of the 46th ACM Technical Symposium on Computer Science Education. New York: Association for Computing Machinery; 2015. p. 522–527. DOI: 10.1145/2676723.2677258.
 30. Qian Y., Lehman J. Students’ misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education*. 2017; 18(1): Article 1. DOI: 10.1145/3077618.
 31. Ettles A., Luxton-Reilly A., Denny P. Common logic errors made by novice programmers. In: Proceedings of the 20th Australasian Computing Education Conference. New York: Association for Computing Machinery; 2018. p. 83–89. DOI: 10.1145/3160489.3160493.
 32. Pea R.D. Language-independent conceptual “bugs” in novice programming. *Journal of Educational Computing Research*. 1986; 2(1): 25–36. DOI: 10.2190/689T-1R2A-X4W4-29J2.
 33. Lister R., Adams E.S., Fitzgerald S., Fone W., Hamer J., Lindholm M., McCartney R., Moström J.E., Sanders K., Seppälä O., Simon B., Thomas L. A multi-national study of reading and tracing skills in novice programmers. *ACM SIGCSE Bulletin*. 2004; 36(4): 119–150. DOI: 10.1145/1041624.1041673.
 34. McCauley R., Fitzgerald S., Lewandowski G., Murphy L., Simon B., Thomas L., Zander C. Debugging: A review of the literature from an educational perspective. *Computer Science Education*. 2008; 18(2): 67–92. DOI: 10.1080/08993400802114581.
 35. Landis J.R., Koch G.G. The measurement of observer agreement for categorical data. *Biometrics*. 1977; 33(1): 159–174. DOI: 10.2307/2529310.

ПРИЛОЖЕНИЯ

Приложение А. Тексты экспериментальных подсказок

Общая подсказка (условие В), пример варианта из 5 пунктов. «Эта программа содержит логическую ошибку. Прежде чем ранжировать весь исходный код, проверьте сначала эти стандартные места сбоя, по порядку: 1) верны ли начальная и конечная границы каждого цикла, включая то, должна ли граница включаться или исключаться; 2) используют ли сравнения и условия правильный оператор и правильное направление, включая строгие и нестрогие сравнения; 3) корректно ли инициализированы и в правильном ли порядке обновляются переменные, накапливающие или отслеживающие состояние; 4) обработаны ли граничные случаи — первый элемент, последний элемент, пустой или односоставный вход; 5) то ли значение выводится или возвращается в конце, которое требует задача. Это гипотезы, а не выводы: проверив их, всё равно взвесьте каждую строку исходного кода по приведённым выше инструкциям».

Подсказка по паттерну (условие С) для класса «полный перебор», 4 пункта. «Эта программа предположительно решает задачу прямой реализацией полного перебора. Прежде чем ранжировать весь исходный код, проверьте сначала эти стандартные места сбоя данного паттерна, по порядку: 1) соответствуют ли границы каждого цикла диапазону, указанному в задаче, включая включение или исключение конечной точки; 2) следует ли код формуле и порядку операций задачи в точности, включая целочисленное и вещественное деление; 3) не переиспользуется ли и не перекрывается ли переменная-индекс вложенного цикла внутренним циклом; 4) выполняется ли финальное накопление или вывод ровно один раз в нужной точке, а не внутри цикла, который его повторяет. Это гипотезы, а не выводы: проверив их, всё равно взвесьте каждую строку исходного кода по приведённым выше инструкциям».

Подсказка по паттерну для класса «динамическое программирование», 5 пунктов — перечисляет: значение и индекс базового случая; учёт рекуррентным соотношением всех предшествующих состояний; согласованность порядка заполнения таблицы с направлением зависимостей; согласованность инициализации таблицы со способом комбинирования результатов; согласованность смещения индексов между условием задачи и индексацией таблицы.

Структура запроса. Каждый запрос состоит из неизменной системной инструкции, шаблона сообщения с пронумерованным по строкам исходным кодом, необязательного блока подсказки (отсутствует в условиях A_G и A_P) и требования вернуть ответ в фиксированном формате — упорядоченный список номеров строк. Единственное различие между условиями — содержимое блока подсказки.

Согласованность пар проверена механически по всем двенадцати классам словаря: число пунктов совпадает точно; расхождение по числу слов не превышает 8 при длине 119–138 слов; вводное предложение, формулировка «проверьте, ...» и заключительная оговорка идентичны по форме; в текстах плацебо не встречается ни один термин словаря паттернов. Приведённые выше тексты — перевод замороженных англоязычных строк, использованных в эксперименте; научным артефактом являются англоязычные оригиналы.

Приложение Б. Полная воронка формирования выборки

Таблица Б1 — От полного корпуса ConDefects-Python до замороженной выборки $n = 30$

Этап	Фильтр	Исключено	Осталось
1	Полный корпус ConDefects-Python (985 задач)	—	2864
2	Синтаксическая ошибка в ошибочной или исправленной версии	1	2863
3	Длина исходника вне диапазона 25–300 строк	1539	1324
4	Исправление затрагивает не одну строку	316	1008
5	Не выполнено условие « ≥ 1 пройденный и ≥ 1 проваленный тест на ошибочной версии; все тесты пройдены на исправленной»	705	303
6	Дедупликация по задаче (одна программа на задачу, посеянный отбор)	71	232
7	Независимая слепая разметка двумя моделями (все 232 размечены обеими)	0	232
8	Точное совпадение меток двух аннотаторов	50	182
9	Согласованная метка вне словаря паттернов	25	157
10	Классы с числом программ менее 7	27	130
11	Классы вне четырёх крупнейших	21	109
12	Посеянный отбор внутри класса (квоты 8/8/7/7)	79	30

Примечание. Исключения на этапе 5 раскладываются как 140 программ, чья ошибочная версия не прошла ни одного теста; 392 — чья ошибочная версия не провалила ни одного теста (ошибка не проявлялась на доступных тестах); 274 — чья исправленная версия не прошла хотя бы один тест. Рамка из 232 программ сформирована до какой-либо разметки паттернов и от неё не зависит. Зерно отбора — 20260910, зафиксировано; повторных розыгрышей не выполнялось.

Приложение В. Дополнительные статистические результаты

Таблица В1 — Точность локализации по всем четырём условиям ($n = 30$ программ, агрегация по правилу большинства из пяти повторений)

Условие	Топ-1	Топ-3	Топ-5	EXAM* (меньше — лучше)
A_G — без блока подсказки (сессия G)	13/30 (43.33%)	17/30 (56.67%)	17/30 (56.67%)	0.452322
A_P — без блока подсказки (сессия P)	12/30 (40.00%)	16/30 (53.33%)	17/30 (56.67%)	0.480003
B — общая подсказка	12/30 (40.00%)	15/30 (50.00%)	16/30 (53.33%)	0.510798
C — подсказка по паттерну	12/30 (40.00%)	18/30 (60.00%)	19/30 (63.33%)	0.445055

Примечание. Значения A_G и A_P приведены раздельно, а не объединены: они образуют нулевой контроль, и их различие — измеряемая величина, а не погрешность. Условие A не является условием «без какого-либо руководства».

Нулевой контроль. A_P – A_G по Топ-1: –3.33 п.п.; дискордантных пар $d = 5$; точный критерий Макнемара $p = 1.0$. Первичное сравнение: $d = 6$. Заранее заявленный критерий интерпретируемости (d нулевого контроля строго меньше d первичного сравнения) выполнен с минимальным запасом.

Детерминизм. Доля ячеек «программа $\times$ условие», в которых все пять повторений дали побайтово идентичный ответ: 0.00% во всех четырёх условиях. Доля ячеек, где совпали все пять исходов Топ-1: A_G 56.67%, A_P 73.33%, B 66.67%, C 66.67%.

Диагностика запросов и ответов. Средняя длина запроса: 762.53 слова (B) против 763.10 слова (C). Средняя длина возвращённого списка кандидатов: 3.02 элемента (B) против 3.08 (C). Отказов разбора ответа: 1 из 600 (условие B), засчитан как промах по заранее зафиксированной политике.

Расчёт объёма подтверждающего исследования. Частота дискордантности составила 0.200 с бутстреп-интервалом [0.067; 0.367]; коэффициент направления $\psi = 0.500$ ровно. При $\psi = 0.5$ знаменатель стандартной аппроксимации для расчёта объёма выборки по

критерию Макнемара обращается в ноль, и расчёт не определён. Зарегистрированный статус: эксперимент не смог определить объём подтверждающего исследования.

Приложение Г. Согласие аннотаторов по классам паттернов

Таблица Г1 — Симметричное согласие по метке (число точных совпадений, делённое на объединение программ, отнесённых к классу хотя бы одним аннотатором)

Класс паттерна	Аннотатор А	Аннотатор В	Точных совпадений	Объединение	Согласие
Обход графа в глубину/ширину	15	13	13	15	86.67%
Динамическое программирование	53	45	45	53	84.91%
Скользящее окно	6	5	5	6	83.33%
«Сортировка, затем проход»	6	8	6	8	75.00%
Полный перебор	50	42	37	55	67.27%
Интервалы	5	5	4	6	66.67%
Жадная стратегия	15	14	11	18	61.11%
Бинарный поиск	23	14	14	23	60.87%
Префиксные суммы	7	9	6	10	60.00%
Пошаговая симуляция	10	20	10	20	50.00%
Вне словаря	29	47	25	51	49.02%
Подсчёт через хеш-таблицу	10	4	4	10	40.00%
Два указателя	3	6	2	7	28.57%

Примечание. Итоговые показатели: 232 сопоставленные программы, 182 точных совпадения, 50 расхождений, необработанное согласие 78.4483%, случайно ожидаемое согласие 13.00%, каппа Коэна ≈ 0.752269 . Согласие сильно связано с уверенностью аннотаторов: при высокой уверенности обеих метки совпали в 97.17% случаев (103 из 106), при низкой уверенности хотя бы одного — в 44–48%.

Приложение Д. Воспроизводимость, конфигурация и заявление об использовании инструментов ИИ

Таблица Д1 — Замороженная конфигурация единственного валидного запуска

Параметр	Значение
Поставщик API	OpenRouter
Модель	z-ai/glm-5.3-flash (идентичность проверялась в каждом ответе)
Нижележащий поставщик вычислений	Z.AI, закреплён; резервные поставщики отключены
Температура	0
Предельный объём ответа	32 768 токенов
Усилие внутреннего рассуждения	low
Повторений на ячейку «программа × условие»	5
Тайм-аут	120 секунд
Параллельных запросов	4
Всего научных обращений	600 (30 программ × 2 сессии × 2 условия × 5 повторений)
Порядок сессий	Сначала G, затем P; определён однократным случайным подбрасыванием до первого обращения

Воспроизводимость. Каждый документ, названный в разделе 2 «замороженным», — это конкретный датированный коммит в линейной истории версий проекта. Выборка снабжена контрольной суммой SHA-256 (9f2db14b...9ef8fc); тексты обеих подсказок заморожены и снабжены контрольными суммами; зерно отбора выборки (20260910) и зерно бутстрепа (20260908) зафиксированы; модель и поставщик закреплёны проверяемым

идентификатором. Каждый сырой ответ модели записывался в отдельный файл до какого-либо разбора, чтобы ошибка разбора или подсчёта не могла незаметно уничтожить свидетельство. Доэкспериментальная заморозка протокола, промптов, модели и плана анализа зафиксирована коммитом `f8c00ff`; запись провенанса внутри итогового файла анализа содержит контрольные суммы выборки, системной инструкции, шаблона сообщения и обоих файлов подсказок, и все они совпадают с этой заморозкой. Финальный запуск полностью сохранён: кросс-сессионный отчёт, операционная сводка, попрограммные таблицы и метрики обеих сессий, содержательный отчёт и полный архив сырых ответов по каждому из 600 обращений.

Операционные показатели финального запуска. Израсходовано 718 300 токенов запроса и 509 495 токенов ответа (из них 429 072 — внутреннее рассуждение, 80 423 — итоговое содержательное поле), всего 1 227 795 токенов. Фактическая стоимость — 0.297 доллара США. Полное время выполнения — 3385.5 секунды (56 минут 25.5 секунды), считая от начала первой сессии до завершения второй, включая проверку согласованности между ними.

Политика «громкого отказа». Зафиксированная операционная политика проекта запрещает тихо повторять деградировавший запуск до получения выглядящих пригодными чисел: это превратило бы сбой конфигурации в нераскрытый отбор запусков. Каждая аннулированная попытка сохранена и диагностирована. Ошибка провенанса меток предыдущей версии выборки также обнаружена, задокументирована и сохранена в истории, а не исправлена задним числом без следа: недействительная версия выборки оставлена неизменной как аудиторский артефакт, а действующая создана как отдельная новая версия.

Заявление об использовании инструментов III. Большие языковые модели использовались в проекте в трёх ролях, каждая из которых раскрывается: (1) как **объект измерения** — модель `z-ai/glm-5.3-flash` является исследуемой системой; (2) как **инструмент разметки** — две модели выполнили независимую слепую разметку алгоритмических паттернов, что раскрыто в разделе 2.7 и учтено как ограничение в разделе 2.15; (3) как **инструмент инженерной реализации и редактирования** при построении конвейера эксперимента и подготовке текста настоящего отчёта на основе зафиксированных артефактов проекта. Ни один результат, направление эффекта или сравнительное утверждение не были порождены редакторским инструментом: каждое числовое значение сверено с зафиксированными в системе контроля версий файлами анализа. В соответствии с правилами конкурсной приемлемости итоговые версии аннотации, исследовательского плана и списка использованных источников, подаваемые на конкурс, должны быть составлены или содержательно проверены самим автором-учащимся.